K8s

KUBERNETES

COURSE SLIDES · NON-WSQ

Kubernetes Fundamentals for Beginners

Course Code: C742

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.0 · 2 October 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer, Tertiary Infotech Academy Pte Ltd

EXPERTISE
Cloud native architecture, Kubernetes, containers, DevOps and cloud solution architecture.

CERTIFICATIONS
AWS Certified Solutions Architect, SysOps Administrator, Cloud Practitioner; ACTA / DACE certified.

DELIVERS
Courses on Kubernetes, cloud architecture, DevOps, data science and applied AI.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

1

Who you are

Your name, your organisation and your role.

2

Your background

Your experience with Linux, containers or Docker, if any.

3

Kubernetes at work

Whether your organisation already runs Kubernetes — and who operates it.

4

Your goal

What you would like to deploy or operate on Kubernetes after this course.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. Attend both days in full.

© 2026 Tertiary Infotech Academy Pte Ltd

LMS / TMS · lms-tms.tertiaryinfotech.com

Download Your Course Material

1

1 · Go to the LMS portal

Open https://lms-tms.tertiaryinfotech.com in your browser.

2

2 · Log in

Sign in with the account e-mail you used to register for this course.

3

3 · Open this course

Select 'Kubernetes Fundamentals for Beginners' (C742).

4

4 · Download

Download the Trainer Slides, Learner Guide and Lesson Plan (PDF).

5

5 · Get the labs

Clone the lab repository from GitHub — the link is on the course page.

6

6 · Keep them open

Refer to these materials during the labs and after the course.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 2 Days, 7.5 Hours per Day

Day 1 — Fundamentals & Containers

- Introductions · course overview · ground rules
- Topic 1: Kubernetes Fundamentals and Architecture
- Labs 1–4 · cluster, the API, workloads and scheduling
- Topic 2 begins · Labs 5–6 · images, Services and Ingress

Day 2 AM — Security & Scale

- Topic 2 continued: security, storage and service mesh
- Labs 7–8 · RBAC, Secrets, NetworkPolicy and storage
- Topic 3: Cloud Native Architecture and Scalability
- Lab 9 autoscaling · Lab 10 resilience

Day 2 PM — Observe & Deliver

- Lab 11 · serverless and open standards
- Topic 4: Labs 12–13 · Prometheus and golden signals
- Topic 5: Labs 14–16 · delivery and GitOps
- Course recap · Q&A · course feedback

© 2026 Tertiary Infotech Academy Pte Ltd

FIVE TOPICS · 16 HANDS-ON LABS

Course Outline

1

Topic 1 — Kubernetes Fundamentals and Architecture

Resources · architecture · the API · containers · scheduling. (46% of the KCNA exam)

2

Topic 2 — Container Orchestration and Kubernetes Operations

Orchestration · runtime · security · networking · service mesh · storage. (22%)

3

Topic 3 — Cloud Native Architecture and Scalability

Autoscaling · serverless · community & governance · roles · open standards. (16%)

4

Topic 4 — Kubernetes Monitoring and Observability

Telemetry & observability · Prometheus · cost management. (8%)

5

Topic 5 — Kubernetes Application Deployment and Management

Application delivery · GitOps · CI/CD. (8%)

© 2026 Tertiary Infotech Academy Pte Ltd

LAB ENVIRONMENT

Set Up Before We Start

1

Docker Desktop

Install Docker Desktop (or Docker Engine on Linux) — it builds the images in Topic 2.

2

minikube

Install minikube to run the local single-node Kubernetes cluster used by every lab.

3

kubectl

Install the kubectl CLI — this is how you talk to the Kubernetes API all course.

4

Helm

Install Helm 3 for the packaging lab in Topic 5.

5

Lab repository

Clone the course labs from GitHub — the link is on your LMS course page.

6

Editor

Use VS Code with the YAML and Kubernetes extensions for editing manifests.

© 2026 Tertiary Infotech Academy Pte Ltd

CORE CONCEPTS

From Containers to Kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

THE PROGRESSION

Why Did Kubernetes Happen?

1

Physical servers

One application per box. Slow to provision, expensive, and mostly idle — utilisation often under 15%.

2

Virtual machines

Many VMs per host improved utilisation, but each carries a full guest operating system and boots in minutes.

3

Containers

Share the host kernel, start in milliseconds, and package the application with its dependencies.

4

Orchestration

Once you run hundreds of containers across many hosts, you need something to schedule and heal them — that is Kubernetes.

© 2026 Tertiary Infotech Academy Pte Ltd

Virtual Machines vs Containers

Dimension	Virtual Machine	Container
Isolation boundary	Hardware virtualisation (hypervisor)	Linux namespaces and cgroups
Operating system	A full guest OS per VM	Shares the host kernel
Typical size	Gigabytes	Tens to hundreds of megabytes
Start-up time	Tens of seconds to minutes	Milliseconds to seconds
Density per host	Tens	Hundreds
Best for	Strong isolation, mixed OS kernels	Elastic, fast-moving microservices

WHEN IT MATTERS Containers are lighter because they do not virtualise hardware — but they share the host kernel, so kernel-level isolation is weaker than a VM's.

© 2026 Tertiary Infotech Academy Pte Ltd

THE CENTRAL IDEA

Kubernetes is a control loop, not a deployment script.

You declare the state you want. Controllers watch the actual state and work continuously to close the gap — forever, not once.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW EVERY CONTROLLER WORKS

The Reconciliation Loop



WHY IT MATTERS

Self-healing is not a feature bolted on — it is what falls out of running this loop forever. Delete a Pod and the loop simply notices the gap and refills it.

© 2026 Tertiary Infotech Academy Pte Ltd

TWO WAYS TO WORK

Declarative vs Imperative

1

Imperative

You issue commands describing HOW: kubectl run, kubectl scale. Fast for experiments, hard to review or repeat.

2

Declarative

You submit a manifest describing WHAT the end state should be: kubectl apply -f. Reviewable, version-controllable, repeatable.

3

Why declarative wins

The manifest in Git is the record of intent — anyone can read it, review it, and reproduce the whole environment from it.

4

The rule for this course

Experiment imperatively if it helps you learn; deliver declaratively. Every lab manifest is a YAML file you keep.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1 · THE REQUEST PATH

Kubernetes Architecture — Control Plane and Nodes



THE KEY INSIGHT

Every component talks only to the API server — never to each other and never directly to etcd. That single front door is what makes the architecture extensible.

© 2026 Tertiary Infotech Academy Pte Ltd

Control-Plane Components

1

kube-apiserver

The front door. Every request — from kubectl, from controllers, from the kubelet — is authenticated, authorised, validated and served here.

2

etcd

A consistent, highly available key-value store holding all cluster state. The only stateful component; back it up or lose the cluster.

3

kube-scheduler

Watches for Pods with no node assigned, filters the nodes that could run them, scores the survivors, and binds the best one.

4

kube-controller-manager

Runs the built-in controllers — Deployment, ReplicaSet, Node, Job, endpoints — each running its own reconciliation loop.

5

cloud-controller-manager

Bridges to the cloud provider for load balancers, routes and node lifecycle when running on a managed platform.

6

Highly available in production

Three or five control-plane nodes with a quorum of etcd members — an even number gains you nothing.

© 2026 Tertiary Infotech Academy Pte Ltd

Worker Node Components

1

kubelet

The node agent. It registers the node, watches the API server for Pods bound to it, and drives the runtime to start containers.

2

Container runtime

containerd or CRI-O, reached through the Container Runtime Interface — it pulls images and runs the containers.

3

kube-proxy

Programmes iptables or IPVS rules so that the virtual IP of a Service reaches a healthy backing Pod.

4

Pods

Your workloads. One or more containers that share a network namespace, an IP address and any mounted volumes.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1 · WORKED EXAMPLE

Reading a Kubernetes Manifest

Every object you write this course has these same four top-level fields — learn to read them once.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: storefront
  namespace: kcna-lab
spec:
  replicas: 3
  selector:
    matchLabels:
      app: storefront
  template:
    metadata:
      labels:
        app: storefront
    spec:
      containers:
      - name: storefront
        image: nginx:1.27
```

apiVersion

Which API group and version validates this object — apps/v1 for Deployments, v1 for core objects.

kind

The type of object. The kind plus apiVersion tells the API server which schema to apply.

metadata

Name, namespace and labels — the identity by which everything else finds this object.

spec

The desired state you are declaring. The controllers write the matching status field back for you.

© 2026 Tertiary Infotech Academy Pte Ltd

The Workload Resources

1

Pod

The smallest deployable unit. Rarely created directly — you almost always let a controller create it for you.

2

ReplicaSet

Keeps exactly N identical Pods running. Managed for you by a Deployment; you rarely write one by hand.

3

Deployment

The workhorse for stateless apps. Manages ReplicaSets to give you rolling updates, rollback and scaling.

4

StatefulSet

For workloads needing stable network identity and their own persistent storage — databases, queues, clustered stores.

5

DaemonSet

Runs exactly one Pod per node. The pattern for log collectors, monitoring agents and CNI plugins.

6

Job and CronJob

Run work to completion, once or on a schedule — batch processing, backups, report generation.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1 · DESIGN DECISION

Which Workload Resource Should You Use?



YES — StatefulSet

Ordinal names (db-0, db-1), a PVC per replica, ordered rollout and a stable DNS name. Databases and clustered stores.

NO — Deployment

Interchangeable replicas, any Pod can serve any request, rolling updates in any order. Almost every stateless service.

If the workload must run on every node instead, the answer is neither — use a DaemonSet. If it must finish and stop, use a Job.

© 2026 Tertiary Infotech Academy Pte Ltd

Labels, Selectors and Namespaces

1

Labels

Key-value pairs on an object: app=storefront, tier=api, env=prod. They carry meaning your tooling can act on.

2

Selectors

How every controller and Service finds its objects. A Service has no list of Pods — it has a selector, matched continuously.

3

Why this matters

Loose coupling: scale a Deployment and the Service picks up the new Pods automatically, because the selector still matches.

4

Namespaces

A logical partition of the cluster for names, quotas and RBAC — dev, staging, prod, or one per team.

5

Not a security boundary alone

Namespaces separate names, not networks. Add NetworkPolicy and RBAC to make the boundary real.

6

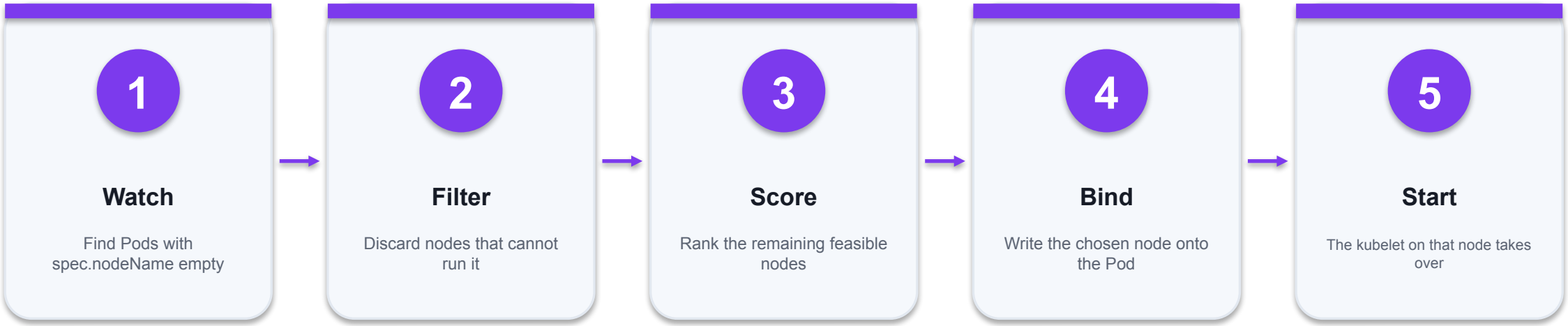
Annotations

Non-identifying metadata for tools — build numbers, checksums, ingress configuration. Never used by selectors.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1 · FILTER THEN SCORE

How the Scheduler Places a Pod



WHY PODS GO PENDING

A Pod stays Pending when filtering leaves no feasible node — insufficient CPU or memory, an unmatched nodeSelector, or an untolerated taint. Always read the FailedScheduling event before guessing.

© 2026 Tertiary Infotech Academy Pte Ltd

Requests vs Limits — What Each One Does

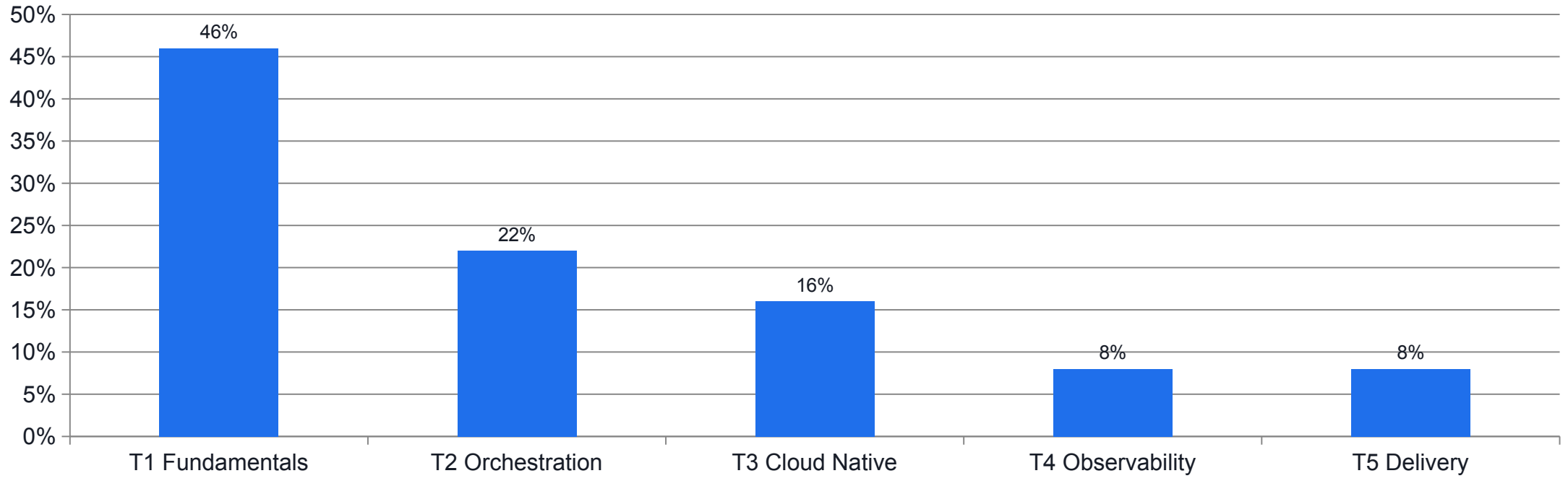
	Request	Limit
Used by	The scheduler	The kubelet and the runtime
Meaning	Guaranteed minimum for placement	Hard ceiling at runtime
If CPU is exceeded	Not applicable	The container is throttled
If memory is exceeded	Not applicable	The container is OOM-killed
Omitting it	The Pod may be scheduled anywhere and starve	The container can consume the whole node

WHEN IT MATTERS Requests determine where a Pod lands; limits determine what happens once it runs. Setting requests without limits is a common and expensive mistake.

© 2026 Tertiary Infotech Academy Pte Ltd

ALIGNED TO THE KCNA EXAM

Where Course Time Goes — Topic Weighting



WHAT THE DATA SHOWS

Topic 1 alone is 46% of the KCNA exam — nearly half. If revision time is short, spend it on architecture, workload resources and scheduling.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 3 PREVIEW

The Cloud Native Landscape

1

CNCF

The Cloud Native Computing Foundation hosts Kubernetes and hundreds of related projects under neutral governance.

2

Maturity levels

Sandbox (early), Incubating (proven adopters), Graduated (production-ready) — a real signal for your blueprint.

3

Graduated examples

Kubernetes, Prometheus, Envoy, containerd, Helm, Argo, Fluentd, etcd, Linkerd, OpenTelemetry.

4

Open standards

OCI (images), CRI (runtimes), CNI (networking), CSI (storage), OpenTelemetry (telemetry) — portability by design.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPICS 1 TO 5

The Journey Through This Course

1. Understand the architecture
2. Orchestrate and secure containers
3. Design for scale and resilience
4. Observe what is actually happening
5. Deliver changes safely and review

© 2026 Tertiary Infotech Academy Pte Ltd

Anatomy of a Dockerfile

Base & setup

- FROM — pick a small, pinned base
- WORKDIR — set the working directory
- COPY / ADD — bring your files in
- Each instruction becomes one layer

Build steps

- RUN — install dependencies
- Each line is one cached layer
- Order steps so the cache is reused
- Slow-changing steps go first

Runtime

- EXPOSE — document the port
- ENV — runtime configuration
- CMD / ENTRYPOINT — what runs
- USER — drop root before running

© 2026 Tertiary Infotech Academy Pte Ltd

Single-Stage vs Multi-Stage Build

	Single stage	Multi-stage
What ships	Compiler, source and dependencies	Only the built artifact
Typical image size	~800 MB	~8 MB
Attack surface	Shell, package manager, build tools	No shell, no package manager
Build speed	One pass	Two stages, cached separately
Use it when	Quick experiments	Anything you ship to production

WHEN IT MATTERS COPY --from=builder pulls just the binary out of the build stage — the compiler and source never reach the final image.

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-Container Pod Patterns

Sidecar

- Runs beside the main application
- Shares an emptyDir volume
- Log shipping, proxying, syncing
- The most common pattern

Init container

- Runs first, to completion
- Pod shows Init:N/M while it runs
- Seed a volume, wait for a dependency
- Must succeed before the app starts

Adapter & Ambassador

- Adapter reshapes output for a consumer
- Ambassador proxies outbound traffic
- Both keep the app code unchanged
- Standardise without rewriting

© 2026 Tertiary Infotech Academy Pte Ltd

Run-to-Completion Workloads

1

completions

How many successful Pod runs the Job needs before it is complete.

2

parallelism

How many Pods the Job may run at the same time.

3

backoffLimit

How many failures are tolerated before the Job is marked failed.

4

activeDeadlineSeconds

A hard wall-clock ceiling that overrides the backoff limit.

5

CronJob schedule

Standard cron syntax plus an optional timeZone field.

6

concurrencyPolicy

Allow, Forbid or Replace — what happens when a run overlaps the previous one.

© 2026 Tertiary Infotech Academy Pte Ltd

Pick the Right Controller

Deployment

- Stateless, interchangeable Pods
- Any number of replicas
- Rolling updates and rollback
- Almost every web service

DaemonSet

- Exactly one Pod per node
- Node-level agents
- Scales with the cluster
- Log collectors, CNI, exporters

StatefulSet

- Stable identity and storage
- Ordered start and stop
- db-0, db-1, db-2 by ordinal
- Databases and queues

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2 · WORKED EXAMPLE

Service Port Mapping — port, targetPort, nodePort

Three port fields confuse almost everyone the first time. Read them once, carefully.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080
```

port

The port clients use to reach the Service on its ClusterIP.

targetPort

The port the container itself is actually listening on.

nodePort

The port opened on every node, from the 30000–32767 range.

selector

How the Service finds its Pods — a label mismatch is the most common bug.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2 · CLUSTER DNS

Service DNS — Names, Not IPs

1

The full name

<service>.<namespace>.svc.cluster.local — the FQDN always resolves from anywhere.

2

Same namespace

Just use the Service name: curl http://web-svc/api — the Pod's search list completes it.

3

Different namespace

Include the namespace: curl http://web-svc.production/api.

4

Who provides it

CoreDNS runs in kube-system and every Pod is configured to resolve through it.

5

Headless Services

clusterIP: None returns every Pod IP instead of one virtual IP — used by StatefulSets.

6

Empty Endpoints

If a Service has no endpoints, the selector does not match any Pod labels. Check that first.

© 2026 Tertiary Infotech Academy Pte Ltd

Ingress — L7 Routing Architecture



SERVICES VS INGRESS

A Service exposes one workload at L4. An Ingress is a single HTTP entry point that routes by host and path to many Services — and terminates TLS on the way in.

© 2026 Tertiary Infotech Academy Pte Ltd

NetworkPolicy Patterns

1

Default is wide open

With no policy, every Pod can reach every other Pod on any port. Isolation is opt-in.

2

Default-deny ingress

podSelector: {} with policyTypes: [Ingress] and no rules blocks all inbound traffic.

3

Default-deny egress

The same shape with policyTypes: [Egress] blocks all outbound traffic.

4

Always allow DNS

Egress policies must permit UDP port 53, or Pods can no longer resolve Service names.

5

Selecting sources

Allow by podSelector (Pods by label), namespaceSelector (a whole namespace) or ipBlock (a CIDR).

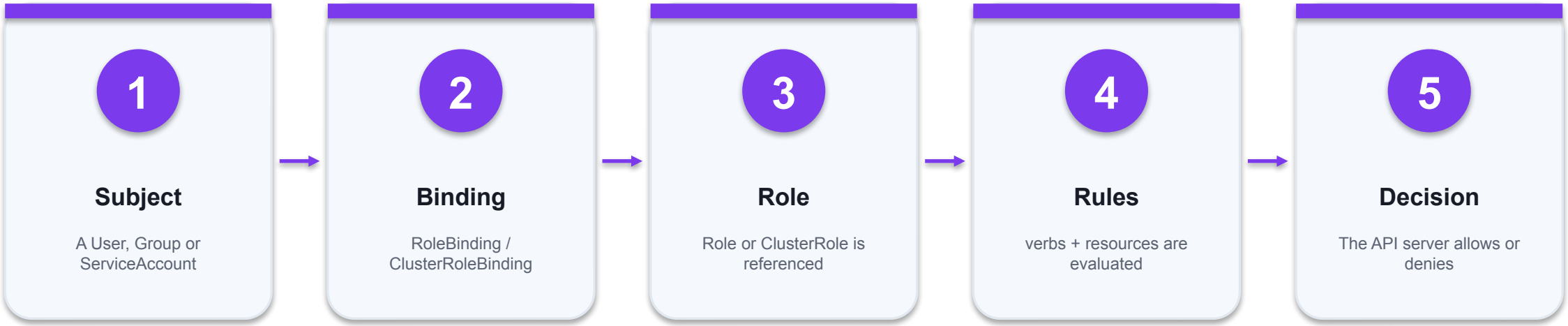
6

Policies are additive

Multiple policies combine with a logical OR of their allow rules — nothing subtracts.

© 2026 Tertiary Infotech Academy Pte Ltd

RBAC — How a Request Is Authorised



SCOPE IS THE TRAP

Role + RoleBinding is namespace-scoped; ClusterRole + ClusterRoleBinding is cluster-wide. A ClusterRole bound with a RoleBinding grants its rules in that one namespace only.

© 2026 Tertiary Infotech Academy Pte Ltd

ConfigMaps and Secrets — Decouple Config from Image

1

ConfigMap

Non-secret key/value data. Create with --from-literal, --from-file or --from-env-file.

2

Secret

The same shape, but base64-encoded and more tightly controlled. Types: generic, tls, docker-registry.

3

Injecting one key

configMapKeyRef or secretKeyRef pulls a single key into an environment variable.

4

Injecting all keys

envFrom with configMapRef or secretRef brings the whole object in at once.

5

Mounting as files

A volume mount writes each key as a file — and volume-mounted ConfigMaps update live.

6

Base64 is not encryption

Anyone who can read the Secret can decode it. Restrict with RBAC and enable encryption at rest.

© 2026 Tertiary Infotech Academy Pte Ltd

Run Containers Safely — securityContext

1

What it sets

The user, group and privileges a container runs with.

2

Two levels

Pod-level applies to every container; container-level overrides it for one.

3

runAsNonRoot: true

Blocks any image whose default user is root — the single highest-value setting.

4

readOnlyRootFilesystem

Makes the container filesystem immutable; writeable paths must be explicit volumes.

5

Drop capabilities

capabilities.drop: [ALL] removes every Linux capability, then add back only what is needed.

6

allowPrivilegeEscalation: false

Stops a process gaining more privileges than its parent had.

© 2026 Tertiary Infotech Academy Pte Ltd

Autoscaling — Three Axes

HPA — Horizontal

- Adds or removes Pod replicas
- Driven by CPU, memory or custom metrics
- Needs metrics-server and resource requests
- The one you will use most

VPA — Vertical

- Right-sizes requests and limits
- Learns from historic usage
- Installed as an add-on
- Do not run it on the same target as an HPA

Cluster Autoscaler

- Adds or removes whole nodes
- Reacts to unschedulable Pods
- Provided by the cloud platform
- The HPA is useless without capacity

© 2026 Tertiary Infotech Academy Pte Ltd

Three Probes, Three Jobs

livenessProbe

- 'Is it still working?'
- Restarts a hung or deadlocked container
- Too aggressive = restart loops
- Handler: httpGet, tcpSocket or exec

readinessProbe

- 'Can it serve traffic yet?'
- Removes the Pod from Service endpoints
- Never restarts the container
- The one that prevents 502s on rollout

startupProbe

- 'Has it finished booting?'
- For slow-starting legacy apps
- Blocks the other two until it passes
- Prevents premature liveness restarts

© 2026 Tertiary Infotech Academy Pte Ltd

Govern Resource Consumption

1

ResourceQuota

Caps what a whole namespace may consume — Pod count, total requests and total limits.

2

LimitRange

Enforces per-container defaults, default requests and maximum values.

3

They work together

In a quota-enforced namespace, a Pod with no resource block is rejected unless a LimitRange supplies defaults.

4

Units

CPU is measured in millicores (600m = 0.6 of a core); memory in bytes (100Mi, 2Gi).

5

Troubleshooting

kubectl describe quota shows Used against Hard — the first place to look.

6

Why it matters

Quotas are how a shared cluster stays fair, and how one team's bug stops being everyone's outage.

© 2026 Tertiary Infotech Academy Pte Ltd

Deployment Strategies

Rolling update

- The default strategy
- maxSurge / maxUnavailable tune it
- Zero downtime with readiness probes
- Old and new run side by side briefly

Recreate & Blue/Green

- Recreate: stop all, then start new
- Causes downtime — but no version overlap
- Blue/green: deploy idle, then switch
- Switch is a Service selector change

Canary

- Small percentage to the new version
- Watch the metrics before committing
- Promote or roll back on evidence
- Lowest blast radius of the four

© 2026 Tertiary Infotech Academy Pte Ltd

01

TOPIC 01

Kubernetes Fundamentals and Architecture

Kubernetes Resources · Architecture · The API · Containers · Scheduling

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 46%

Key Concepts — Kubernetes Fundamentals and Architecture

1. Kubernetes is a declarative system: you submit the desired state as an object, and controllers work continuously to make the actual state match it.

2. The control plane (kube-apiserver, etcd, kube-scheduler, kube-controller-manager) decides what should run; the worker nodes (kubelet, kube-proxy, container runtime) actually run it.

3. The kube-apiserver is the single front door — every component, including kubectl, talks only to the API server, never directly to etcd or to each other.

4. etcd is the only stateful component: a consistent, highly available key-value store holding the entire cluster state. Losing etcd means losing the cluster.

5. A Pod is the smallest deployable unit — one or more containers sharing a network namespace, an IP address and storage volumes.

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 46%

Key Concepts — Kubernetes Fundamentals and Architecture (continued)

1. Workload resources build on Pods: a ReplicaSet keeps N replicas alive, a Deployment manages ReplicaSets to give rolling updates and rollback.

2. DaemonSets run one Pod per node (agents), StatefulSets give stable identity and storage (databases), Jobs and CronJobs run work to completion.

3. Namespaces partition a cluster logically; labels and selectors are how every controller and Service finds the objects it manages.

4. The scheduler places a Pod on a node by filtering (which nodes *can* run it) then scoring (which node is *best*), honouring resource requests, taints, tolerations and affinity.

5. Resource requests drive scheduling and limits drive enforcement — a container exceeding its memory limit is OOM-killed; exceeding its CPU limit is throttled.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Kubernetes Fundamentals and Architecture

Labs 1–2

- Bootstrap a Kubernetes Cluster and Explore the Architecture
- Work with the Kubernetes API and kubectl

Labs 3–4

- Deploy and Manage Pods, ReplicaSets and Deployments
- Control Pod Scheduling with Requests, Taints and Affinity

—

- —

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Bootstrap a Kubernetes Cluster and Explore the Architecture

LAB 1

Explain the Kubernetes control-plane and node components and verify each one in a running cluster.

The learner starts a local Kubernetes cluster with minikube, inspects every control-plane and node component, and maps what they observe onto the reference architecture diagram used in the proof of concept.

YOU'LL BUILD

A running single-node Kubernetes cluster plus an annotated architecture inventory of its components

TOOLCHAIN

minikube, kubectl, kube-apiserver, etcd, kube-scheduler, kube-controller-manager, kubelet, kube-proxy

DONE WHEN

kubectl get nodes shows STATUS=Ready, kubectl get pods -n kube-system shows kube-apiserver, etcd, kube-scheduler, kube-controller-manager and kube-proxy all Running, and your architecture-inventory.md names the role of each.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 1 — Files in Your Lab Folder

1

architecture-inventory.md

Worksheet: record each component, where it runs and what it does.

2

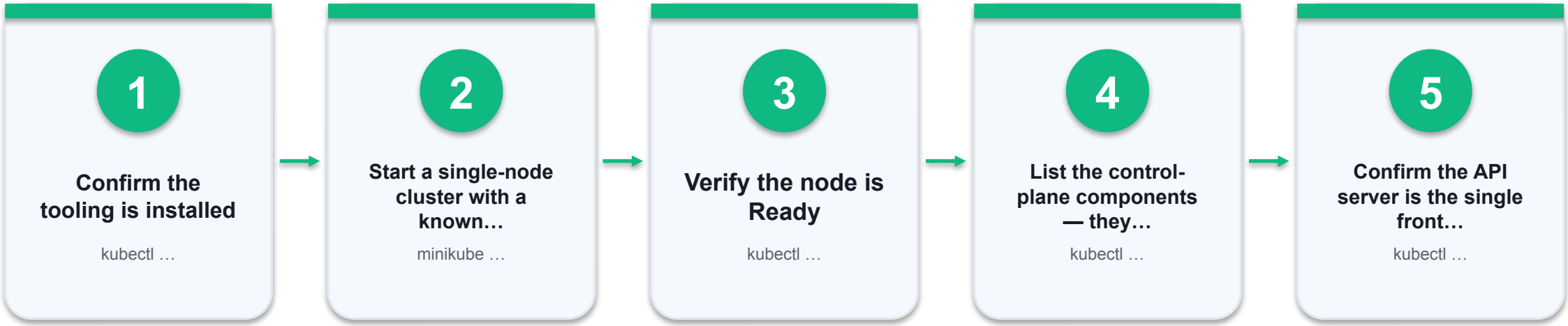
cluster-info.sh

Helper script that dumps the component inventory in one pass.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCESS MAP

How Lab 1 Runs



THE POINT

Explain the Kubernetes control-plane and node components and verify each one in a running cluster.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCEDURE

Bootstrap a Kubernetes Cluster and Explore the Architecture — Part 1/3

1

Confirm the tooling is installed and check the client version

```
$ kubectl version --client && minikube version
```

2

Start a single-node cluster with a known Kubernetes version

```
$ minikube start --nodes=1 --cpus=2 --memory=4096 --kubernetes-version=v1.31.0
```

3

Verify the node is Ready and inspect its capacity and conditions

```
$ kubectl get nodes -o wide && kubectl describe node minikube | head -40
```

4

List the control-plane components — they run as static Pods in kube-system

```
$ kubectl get pods -n kube-system -o wide
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCEDURE

Bootstrap a Kubernetes Cluster and Explore the Architecture — Part 2/3

5

Confirm the API server is the single front door and check cluster health

```
$ kubectl cluster-info && kubectl get --raw='/readyz?verbose' | head -20
```

6

Inspect the etcd static Pod manifest to see how the datastore is configured

```
$ kubectl -n kube-system describe pod -l component=etcd | head -30
```

7

Watch the scheduler and controller-manager leader election in the logs

```
$ kubectl -n kube-system logs -l component=kube-scheduler --tail=20
```

8

Inspect kubelet on the node — the agent that actually starts containers

```
$ minikube ssh -- 'sudo systemctl status kubelet --no-pager | head -15'
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCEDURE

Bootstrap a Kubernetes Cluster and Explore the Architecture — Part 3/3

9

List every API resource the cluster serves, grouped by API group

```
$ kubectl api-resources --sort-by=name | head -40
```

10

Complete architecture-inventory.md, recording each component and its role

```
$ bash cluster-info.sh > my-cluster-inventory.txt
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · VERIFY

Bootstrap a Kubernetes Cluster and Explore the Architecture

✅ Expected result

kubectl get nodes shows STATUS=Ready, kubectl get pods -n kube-system shows kube-apiserver, etcd, kube-scheduler, kube-controller-manager and kube-proxy all Running, and your architecture-inventory.md names the role of each.

IF IT DOESN'T WORK

minikube start fails

Give Docker more resources, then: minikube delete && minikube start --cpus=2 --memory=4096.

Node stays NotReady

The CNI is still starting. Wait 60s, then kubectl get pods -n kube-system.

kubectl cannot connect

The context is wrong: kubectl config use-context minikube.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Work with the Kubernetes API and kubectl

LAB 2

Use the declarative Kubernetes API to create, inspect and delete objects, and read the API directly.

The learner drives the Kubernetes API through kubectl and through raw HTTP, comparing imperative and declarative object management, and reads the anatomy of an API object — apiVersion, kind, metadata, spec and status.

YOU'LL BUILD

A namespace and Pod created declaratively, plus a raw API call transcript showing the same objects

TOOLCHAIN

kubectl, kube-apiserver, kubectl proxy, YAML manifests, API groups and versions

DONE WHEN

curl against /api/v1/namespaces/kcna-lab/pods/nginx returns the same object that kubectl get pod prints, confirming kubectl is only a client of the API server.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 2 — Files in Your Lab Folder

1

namespace.yaml

The kcna-lab namespace used by every later lab.

2

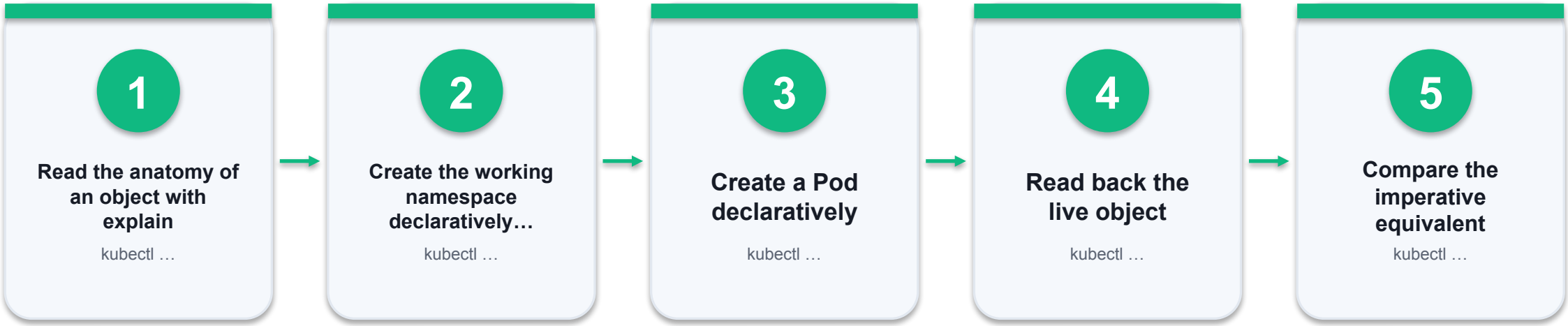
pod-nginx.yaml

A minimal Pod manifest showing apiVersion/kind/metadata/spec.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCESS MAP

How Lab 2 Runs



THE POINT

Use the declarative Kubernetes API to create, inspect and delete objects, and read the API directly.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCEDURE

Work with the Kubernetes API and kubectl — Part 1/3

1

Read the anatomy of an object with explain, field by field

```
$ kubectl explain pod.spec.containers --recursive | head -30
```

2

Create the working namespace declaratively from the manifest

```
$ kubectl apply -f namespace.yaml
```

3

Create a Pod declaratively and watch it move through its phases

```
$ kubectl apply -f pod-nginx.yaml && kubectl get pod -n kcna-lab -w --timeout=60s
```

4

Read back the live object, including the status the controllers wrote

```
$ kubectl get pod nginx -n kcna-lab -o yaml | head -40
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCEDURE

Work with the Kubernetes API and kubectl — Part 2/3

5 Compare the imperative equivalent, using --dry-run to see the generated object

```
$ kubectl run temp --image=nginx:1.27 --dry-run=client -o yaml
```

6 Open a local proxy to the API server and call the REST endpoint directly

```
$ kubectl proxy --port=8001 &
```

7 Query the same Pod over raw HTTP to prove kubectl is only an API client

```
$ curl -s http://127.0.0.1:8001/api/v1/namespaces/kcna-lab/pods/nginx | head -30
```

8 Inspect the API groups and versions the server exposes

```
$ curl -s http://127.0.0.1:8001/apis | head -30
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCEDURE

Work with the Kubernetes API and kubectl — Part 3/3

9 Use a label selector and JSONPath to extract just the fields you need

```
$ kubectl get pods -n kcna-lab -l app=nginx -o jsonpath='{.items[*].status.podIP}'
```

10 Clean up the proxy process when you are finished

```
$ kill %1
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · VERIFY

Work with the Kubernetes API and kubectl

✅ Expected result

curl against /api/v1/namespaces/kcna-lab/pods/nginx returns the same object that kubectl get pod prints, confirming kubectl is only a client of the API server.

IF IT DOESN'T WORK

Namespace not found

Apply it first: kubectl apply -f namespace.yaml.

curl to the proxy refused

kubectl proxy is not running. Start it and keep that shell open.

explain returns nothing

Check the resource path spelling: kubectl explain pod.spec.containers.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Deploy and Manage Pods, ReplicaSets and Deployments

LAB 3

Build the core workload resources and explain how a Deployment manages ReplicaSets which manage Pods.

The learner deploys the sample store-front application, scales it, breaks it deliberately to observe self-healing, then performs a rolling update and a rollback to see the Deployment/ReplicaSet/Pod ownership chain in action.

YOU'LL BUILD

A 3-replica Deployment that survives Pod deletion, plus a completed rolling update and rollback

TOOLCHAIN

Deployment, ReplicaSet, Pod, kubectl scale, kubectl rollout, labels and selectors

DONE WHEN

After deleting a Pod the replica count returns to the desired number within seconds; after the rollback, kubectl describe deployment shows the original nginx:1.27 image and two ReplicaSets exist with only the original scaled up.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 3 — Files in Your Lab Folder

1

deployment-storefront.yaml

3-replica Deployment of the sample store-front app.

2

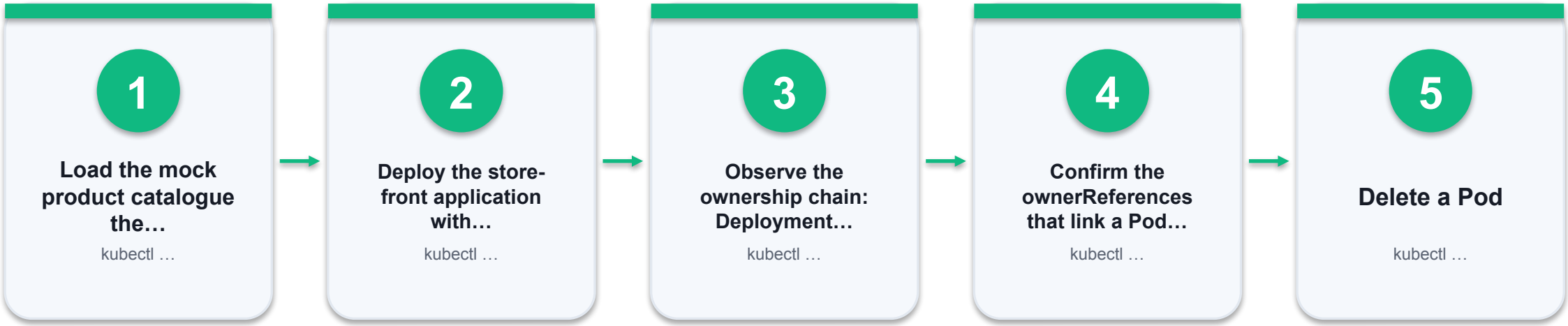
configmap-catalog.yaml

Mock product catalogue (10 products) served by the app.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCESS MAP

How Lab 3 Runs



THE POINT

Build the core workload resources and explain how a Deployment manages ReplicaSets which manage Pods.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCEDURE

Deploy and Manage Pods, ReplicaSets and Deployments — Part 1/3

1

Load the mock product catalogue the application will serve

```
$ kubectl apply -f configmap-catalog.yaml
```

2

Deploy the store-front application with three replicas

```
$ kubectl apply -f deployment-storefront.yaml
```

3

Observe the ownership chain: Deployment owns ReplicaSet owns Pods

```
$ kubectl get deploy,rs,pods -n kcna-lab -l app=storefront
```

4

Confirm the ownerReferences that link a Pod back to its ReplicaSet

```
$ kubectl get pods -n kcna-lab -l app=storefront -o jsonpath='{.items[0].metadata.ownerReferenc...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCEDURE

Deploy and Manage Pods, ReplicaSets and Deployments — Part 2/3

5

Delete a Pod and watch the ReplicaSet controller replace it (self-healing)

```
$ kubectl delete pod -n kcna-lab -l app=storefront --field-selector=status.phase=Running --wait...
```

6

Scale the Deployment up and confirm the new replica count

```
$ kubectl scale deployment storefront -n kcna-lab --replicas=5
```

7

Perform a rolling update to a new image version

```
$ kubectl set image deployment/storefront storefront=nginx:1.27-alpine -n kcna-lab
```

8

Watch the rollout progress and see a second ReplicaSet appear

```
$ kubectl rollout status deployment/storefront -n kcna-lab && kubectl get rs -n kcna-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCEDURE

Deploy and Manage Pods, ReplicaSets and Deployments — Part 3/3

9 Review the revision history the Deployment keeps for you

```
$ kubectl rollout history deployment/storefront -n kcna-lab
```

10 Roll back to the previous revision and confirm the image reverted

```
$ kubectl rollout undo deployment/storefront -n kcna-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · VERIFY

Deploy and Manage Pods, ReplicaSets and Deployments

✅ Expected result

After deleting a Pod the replica count returns to the desired number within seconds; after the rollback, kubectl describe deployment shows the original nginx:1.27 image and two ReplicaSets exist with only the original scaled up.

IF IT DOESN'T WORK

Pods never become Ready

Read the events: kubectl describe pod -n kcna-lab -l app=storefront.

Deleted Pod not replaced

You deleted the Deployment, not the Pod. Re-apply the manifest.

Rollback changes nothing

Only one revision exists. Check kubectl rollout history first.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Control Pod Scheduling with Requests, Taints and Affinity

LAB 4

Influence where the scheduler places a Pod using resource requests, node selectors, taints, tolerations and affinity.

The learner works through the scheduler's filter-and-score decision: setting resource requests that make a Pod unschedulable, labelling nodes, applying taints and tolerations, and expressing node and Pod affinity rules.

YOU'LL BUILD

A set of Pods demonstrating successful placement, a deliberate Pending Pod, and a toleration that restores scheduling

TOOLCHAIN

kube-scheduler, resource requests and limits, nodeSelector, taints and tolerations, node affinity, pod anti-affinity

DONE WHEN

unschedulable-demo stays Pending with a FailedScheduling event naming Insufficient memory, while toleration-demo schedules successfully onto the tainted node — proving the filter stage of the scheduler.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 4 — Files in Your Lab Folder

1. **pod-resources.yaml**
   Pod with explicit CPU/memory requests and limits.

2. **pod-unschedulable.yaml**
   Pod requesting more memory than any node has — stays Pending.

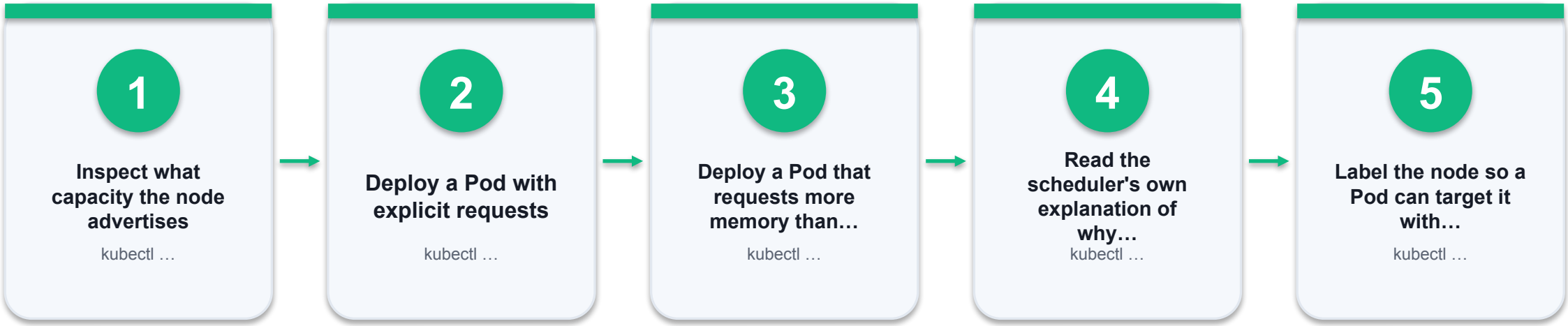
3. **pod-toleration.yaml**
   Pod carrying the toleration for the tainted node.

4. **deployment-antiaffinity.yaml**
   Deployment spreading replicas with pod anti-affinity.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCESS MAP

How Lab 4 Runs



THE POINT

Influence where the scheduler places a Pod using resource requests, node selectors, taints, tolerations and affinity.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCEDURE

Control Pod Scheduling with Requests, Taints and Affinity — Part 1/3

1

Inspect what capacity the node advertises and what is already requested

```
$ kubectl describe node minikube | grep -A8 'Allocated resources'
```

2

Deploy a Pod with explicit requests and limits and confirm placement

```
$ kubectl apply -f pod-resources.yaml && kubectl get pod resources-demo -n kcna-lab -o wide
```

3

Deploy a Pod that requests more memory than the node has

```
$ kubectl apply -f pod-unschedulable.yaml
```

4

Read the scheduler's own explanation of why the Pod is Pending

```
$ kubectl describe pod unschedulable-demo -n kcna-lab | grep -A5 Events
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCEDURE

Control Pod Scheduling with Requests, Taints and Affinity — Part 2/3

5

Label the node so a Pod can target it with a nodeSelector

```
$ kubectl label node minikube disktype=ssd --overwrite
```

6

Taint the node so that only tolerating Pods may be scheduled onto it

```
$ kubectl taint node minikube workload=restricted:NoSchedule --overwrite
```

7

Deploy a Pod carrying the matching toleration and confirm it schedules

```
$ kubectl apply -f pod-toleration.yaml && kubectl get pod toleration-demo -n kcna-lab -o wide
```

8

Deploy a Deployment using pod anti-affinity to spread replicas apart

```
$ kubectl apply -f deployment-antiaffinity.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCEDURE

Control Pod Scheduling with Requests, Taints and Affinity — Part 3/3

9 Observe how anti-affinity affects placement on a single-node cluster

```
$ kubectl get pods -n kcna-lab -l app=spread -o wide
```

10 Remove the taint to restore normal scheduling before the next lab

```
$ kubectl taint node minikube workload=restricted:NoSchedule-
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · VERIFY

Control Pod Scheduling with Requests, Taints and Affinity

✅ Expected result

unschedulable-demo stays Pending with a FailedScheduling event naming Insufficient memory, while toleration-demo schedules successfully onto the tainted node — proving the filter stage of the scheduler.

IF IT DOESN'T WORK

Pod stuck in Pending

Expected for unschedulable-demo. Read the FailedScheduling event.

Toleration Pod also Pending

The node label is missing: kubectl label node minikube disktype=ssd.

Taint will not clear

The trailing minus removes it: kubectl taint node minikube workload=restricted:NoSchedule-.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Kubernetes Fundamentals and Architecture

- You can now: Explain the Kubernetes control-plane and node components and verify each one in a running cluster.
- You can now: Use the declarative Kubernetes API to create, inspect and delete objects, and read the API directly.
- You can now: Build the core workload resources and explain how a Deployment manages ReplicaSets which manage Pods.
- You can now: Influence where the scheduler places a Pod using resource requests, node selectors, taints, tolerations and affinity.

© 2026 Tertiary Infotech Academy Pte Ltd

02

TOPIC 02

Container Orchestration and Kubernetes Operations

Orchestration Fundamentals · Runtime · Security · Networking · Service Mesh · Storage

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 22%

Key Concepts — Container Orchestration and Kubernetes Operations

1. A container is a process isolated by Linux namespaces and constrained by cgroups — it is not a virtual machine and shares the host kernel.

2. Orchestration adds what a single container runtime cannot: scheduling across many hosts, self-healing, service discovery, scaling and rolling updates.

3. The Container Runtime Interface (CRI) lets Kubernetes run containerd or CRI-O interchangeably; the OCI standard defines the image and runtime formats.

4. The Kubernetes network model requires that every Pod gets its own IP and that all Pods can reach each other without NAT — the CNI plugin implements this.

5. A Service gives a stable virtual IP and DNS name in front of an ephemeral set of Pods: ClusterIP (internal), NodePort, LoadBalancer, and Ingress for HTTP routing.

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 22%

Key Concepts — Container Orchestration and Kubernetes Operations (continued)

1. NetworkPolicies are the Pod-level firewall — by default all traffic is allowed, and a policy switches the selected Pods to default-deny.

2. Security is layered as the 4C model: Cloud, Cluster, Container, Code. RBAC, ServiceAccounts, Secrets and Pod Security Standards each cover a different layer.

3. RBAC binds a subject (user, group or ServiceAccount) to a Role (namespaced) or ClusterRole (cluster-wide) through a RoleBinding or ClusterRoleBinding.

4. A service mesh (Istio, Linkerd) adds mTLS, retries, traffic splitting and per-request telemetry through sidecar proxies, without changing application code.

5. Storage is decoupled: a PersistentVolumeClaim requests storage, a StorageClass provisions a PersistentVolume dynamically, and the CSI driver does the work.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Container Orchestration and Kubernetes Operations

Labs 5–6

- Build and Run Containers with Docker and the OCI Standard
- Expose Applications with Services and Ingress

Labs 7–8

- Secure the Cluster with RBAC, Secrets and NetworkPolicy
- Persist Data with PersistentVolumes, StatefulSets and Storage Classes

—

- —

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Build and Run Containers with Docker and the OCI Standard

LAB 5

Build an OCI image, run it as a container, and explain the runtime layer beneath Kubernetes.

The learner containerises the sample inventory API, inspects the resulting OCI image layers, runs it locally with resource constraints, and then confirms which CRI runtime the Kubernetes cluster is actually using.

YOU'LL BUILD

A tagged OCI image of the inventory API, running locally and loaded into the cluster

TOOLCHAIN

Docker, OCI image spec, containerd, CRI, crictl, image layers and registries

DONE WHEN

curl http://localhost:8080/inventory returns the 12 mock SKUs, and the node's containerRuntimeVersion reports containerd (or cri-o), confirming Kubernetes runs OCI images through the CRI.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 5 — Files in Your Lab Folder

1. **Dockerfile**
   Multi-stage build for the inventory API.

2. **app.py**
   Flask inventory API serving the mock inventory data.

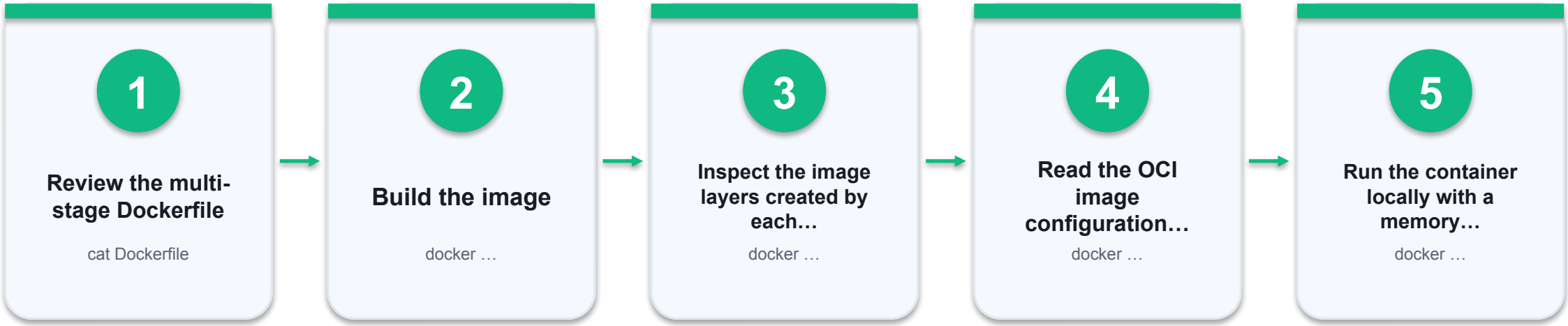
3. **requirements.txt**
   Python dependencies for the API.

4. **inventory.json**
   Mock inventory data — 12 SKUs with stock levels.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCESS MAP

How Lab 5 Runs



THE POINT

Build an OCI image, run it as a container, and explain the runtime layer beneath Kubernetes.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCEDURE

Build and Run Containers with Docker and the OCI Standard — Part 1/3

1

Review the multi-stage Dockerfile and note why the runtime stage is slim

```
$ cat Dockerfile
```

2

Build the image and tag it with a version

```
$ docker build -t inventory-api:1.0 .
```

3

Inspect the image layers created by each Dockerfile instruction

```
$ docker history inventory-api:1.0
```

4

Read the OCI image configuration — entrypoint, env and architecture

```
$ docker inspect inventory-api:1.0 --format '{{json .Config}}' | python3 -m json.tool | head -20
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCEDURE

Build and Run Containers with Docker and the OCI Standard — Part 2/3

5 Run the container locally with a memory limit and a published port

```
$ docker run -d --name inv --memory=256m -p 8080:8080 inventory-api:1.0
```

6 Call the API and confirm the mock inventory data is served

```
$ curl -s http://localhost:8080/inventory | head -20
```

7 Show that the container is just an isolated process on the host

```
$ docker top inv && docker stats --no-stream inv
```

8 Stop and remove the local container

```
$ docker rm -f inv
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCEDURE

Build and Run Containers with Docker and the OCI Standard — Part 3/3

9 Load the image into the cluster so Kubernetes can run it without a registry

```
$ minikube image load inventory-api:1.0
```

10 Confirm which CRI runtime the node uses to run containers

```
$ kubectl get node minikube -o jsonpath='{.status.nodeInfo.containerRuntimeVersion}'
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · VERIFY

Build and Run Containers with Docker and the OCI Standard

✅ Expected result

curl http://localhost:8080/inventory returns the 12 mock SKUs, and the node's containerRuntimeVersion reports containerd (or cri-o), confirming Kubernetes runs OCI images through the CRI.

IF IT DOESN'T WORK

docker build fails

Run it from the lab folder so Dockerfile and app.py are in the build context.

Port 8080 already in use

Stop the other container: docker rm -f inv, or publish a different host port.

Image not found in cluster

Side-load it: minikube image load inventory-api:1.0.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Expose Applications with Services and Ingress

LAB 6

Apply Kubernetes networking to give stable discovery and external access to ephemeral Pods.

The learner deploys the inventory API, fronts it with a ClusterIP Service, proves DNS-based service discovery from inside the cluster, then exposes it externally through NodePort and an Ingress with host-based routing.

YOU'LL BUILD

An internally discoverable ClusterIP Service and an externally reachable Ingress route

TOOLCHAIN

Service (ClusterIP/NodePort/LoadBalancer), CoreDNS, Endpoints, Ingress, ingress-nginx, CNI

DONE WHEN

The curl-test Pod resolves inventory.kcna-lab.svc.cluster.local and returns inventory JSON, and the same data is returned through the Ingress using the Host header — proving discovery works internally and externally.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 6 — Files in Your Lab Folder

1. **deployment-inventory.yaml**
   Deployment of the inventory API built in Lab 5.

2. **service-clusterip.yaml**
   ClusterIP Service selecting the inventory Pods.

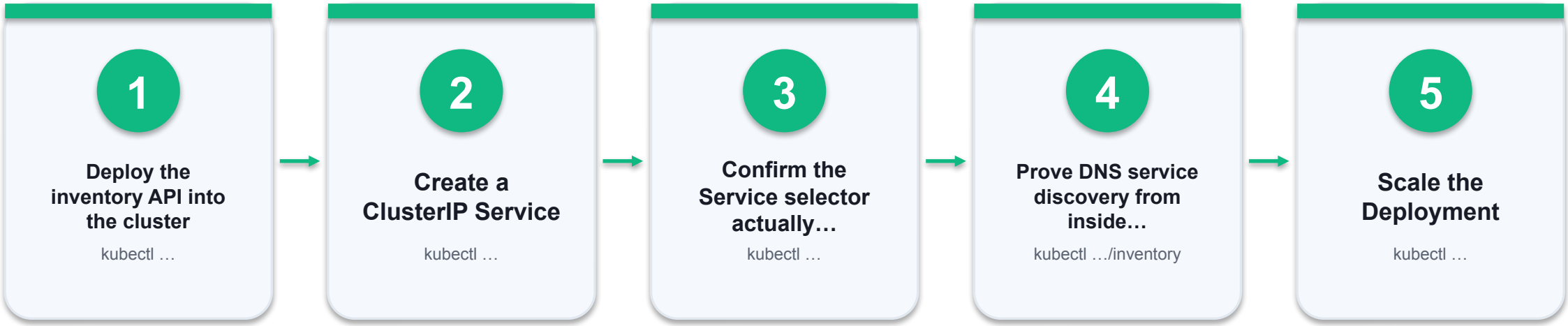
3. **service-nodeport.yaml**
   NodePort Service for external access.

4. **ingress-inventory.yaml**
   Ingress with host-based routing for inventory.kcna.local.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCESS MAP

How Lab 6 Runs



THE POINT

Apply Kubernetes networking to give stable discovery and external access to ephemeral Pods.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCEDURE

Expose Applications with Services and Ingress — Part 1/3

1

Deploy the inventory API into the cluster

```
$ kubectl apply -f deployment-inventory.yaml
```

2

Create a ClusterIP Service and inspect the virtual IP it receives

```
$ kubectl apply -f service-clusterip.yaml && kubectl get svc inventory -n kcna-lab
```

3

Confirm the Service selector actually matched Pods by reading its Endpoints

```
$ kubectl get endpoints inventory -n kcna-lab -o wide
```

4

Prove DNS service discovery from inside another Pod

```
$ kubectl run curl-test -n kcna-lab --rm -it --image=curlimages/curl --restart=Never -- curl -s...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCEDURE

Expose Applications with Services and Ingress — Part 2/3

5

Scale the Deployment and watch the Endpoints list grow automatically

```
$ kubectl scale deploy inventory -n kcna-lab --replicas=3 && kubectl get endpoints inventory -n...
```

6

Expose the same Pods externally with a NodePort Service

```
$ kubectl apply -f service-nodeport.yaml
```

7

Reach the NodePort from outside the cluster

```
$ curl -s $(minikube ip):30080/inventory | head -10
```

8

Enable the ingress controller that will terminate HTTP routing

```
$ minikube addons enable ingress
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCEDURE

Expose Applications with Services and Ingress — Part 3/3

9 Create the Ingress with host-based routing to the Service

```
$ kubectl apply -f ingress-inventory.yaml && kubectl get ingress -n kcna-lab
```

10 Test the host-based route through the ingress controller

```
$ curl -s -H 'Host: inventory.kcna.local' http://$(minikube ip)/inventory | head -10
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · VERIFY

Expose Applications with Services and Ingress

✅ Expected result

The curl-test Pod resolves inventory.kcna-lab.svc.cluster.local and returns inventory JSON, and the same data is returned through the Ingress using the Host header — proving discovery works internally and externally.

IF IT DOESN'T WORK

Service has no endpoints

The selector does not match the Pod labels — the most common Service bug.

Ingress returns 404

Enable the controller: minikube addons enable ingress, then wait for it to be Ready.

Host routing fails

Pass the header: curl -H 'Host: inventory.kcna.local' http://$(minikube ip)/.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Secure the Cluster with RBAC, Secrets and NetworkPolicy

LAB 7

Apply the 4C security model using RBAC, ServiceAccounts, Secrets and network segmentation.

The learner creates a least-privilege ServiceAccount bound to a namespaced Role, verifies the permission boundary with kubectl auth can-i, mounts credentials from a Secret, and switches the namespace to default-deny with a NetworkPolicy.

YOU'LL BUILD

A least-privilege ServiceAccount, a mounted Secret, and a default-deny network posture with one allowed path

TOOLCHAIN

RBAC (Role, RoleBinding), ServiceAccount, Secret, NetworkPolicy, Pod Security Standards

DONE WHEN

auth can-i returns yes for list pods and no for delete pods and for kube-system; after the default-deny policy an unlabelled Pod cannot reach the inventory Service, while a Pod labelled app=frontend can.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 7 — Files in Your Lab Folder

1. **serviceaccount.yaml**
   The inventory-reader ServiceAccount.

2. **role-rbac.yaml**
   Role + RoleBinding granting read-only access to Pods.

3. **secret-dbcreds.yaml**
   Mock database credentials as a Secret.

4. **pod-secret-consumer.yaml**
   Pod mounting the Secret as environment variables.

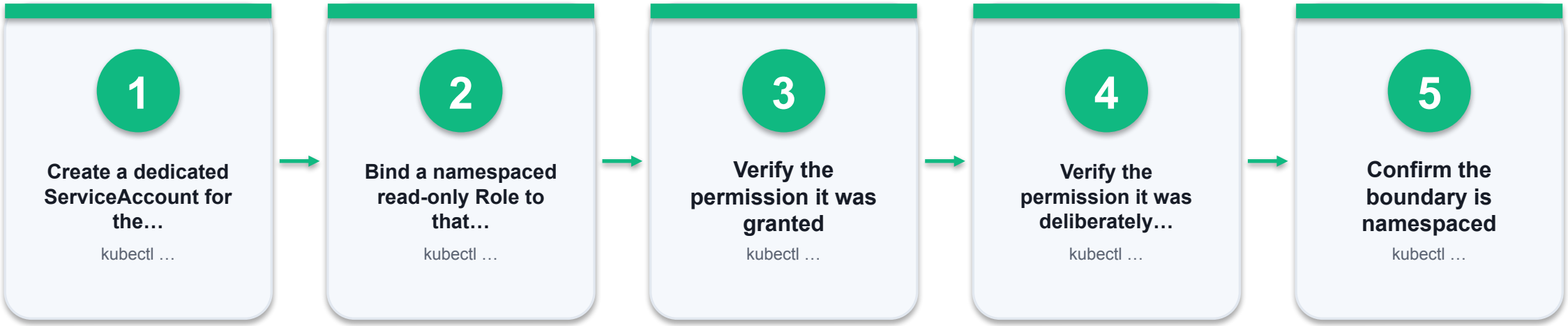
5. **networkpolicy-default-deny.yaml**
   Default-deny ingress policy for the namespace.

6. **networkpolicy-allow-frontend.yaml**
   Policy allowing only the frontend to reach the API.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCESS MAP

How Lab 7 Runs



THE POINT

Apply the 4C security model using RBAC, ServiceAccounts, Secrets and network segmentation.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCEDURE

Secure the Cluster with RBAC, Secrets and NetworkPolicy — Part 1/3

1

Create a dedicated ServiceAccount for the application

```
$ kubectl apply -f serviceaccount.yaml
```

2

Bind a namespaced read-only Role to that ServiceAccount

```
$ kubectl apply -f role-rbac.yaml
```

3

Verify the permission it was granted

```
$ kubectl auth can-i list pods -n kcna-lab --as=system:serviceaccount:kcna-lab:inventory-reader
```

4

Verify the permission it was deliberately NOT granted

```
$ kubectl auth can-i delete pods -n kcna-lab --as=system:serviceaccount:kcna-lab:inventory-reader
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCEDURE

Secure the Cluster with RBAC, Secrets and NetworkPolicy — Part 2/3

5

Confirm the boundary is namespaced, not cluster-wide

```
$ kubectl auth can-i list pods -n kube-system --as=system:serviceaccount:kcna-lab:inventory-reader
```

6

Create the mock database credentials as a Secret

```
$ kubectl apply -f secret-dbcreds.yaml
```

7

Show that a Secret is base64-encoded, not encrypted at rest by default

```
$ kubectl get secret db-credentials -n kcna-lab -o jsonpath='{.data.password}' | base64 -d
```

8

Run a Pod that consumes the Secret as environment variables

```
$ kubectl apply -f pod-secret-consumer.yaml && kubectl logs secret-consumer -n kcna-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCEDURE

Secure the Cluster with RBAC, Secrets and NetworkPolicy — Part 3/3

9 Apply a default-deny NetworkPolicy and confirm traffic is now blocked

```
$ kubectl apply -f networkpolicy-default-deny.yaml
```

10 Allow only the frontend to reach the API, then re-test connectivity

```
$ kubectl apply -f networkpolicy-allow-frontend.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · VERIFY

Secure the Cluster with RBAC, Secrets and NetworkPolicy

✅ Expected result

auth can-i returns yes for list pods and no for delete pods and for kube-system; after the default-deny policy an unlabelled Pod cannot reach the inventory Service, while a Pod labelled app=frontend can.

IF IT DOESN'T WORK

auth can-i always says yes

You are testing as yourself. Add --as=system:serviceaccount:kcna-lab:inventory-reader.

Secret value looks garbled

It is base64. Decode it: kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 -d.

NetworkPolicy has no effect

The CNI must enforce policy. On minikube start with --cni=calico.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Persist Data with PersistentVolumes, StatefulSets and Storage Classes

LAB 8

Decouple storage from compute using PVCs, StorageClasses and StatefulSets, and model a service mesh's role.

The learner provisions storage dynamically through a StorageClass, proves data survives Pod deletion, deploys a StatefulSet with per-replica volumes and stable identities, and compares this with the sidecar model used by a service mesh.

YOU'LL BUILD

A dynamically provisioned PVC whose data survives Pod deletion, plus a 3-replica StatefulSet with stable storage

TOOLCHAIN

PersistentVolume, PersistentVolumeClaim, StorageClass, CSI, StatefulSet, headless Service, service mesh concepts

DONE WHEN

The order records survive deleting and recreating the Pod, and the StatefulSet produces database-0/1/2 each with its own PVC, with database-0 resolvable by its stable DNS name.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 8 — Files in Your Lab Folder

1. **pvc-orders.yaml**
   PersistentVolumeClaim requesting 1Gi of storage.

2. **pod-with-pvc.yaml**
   Pod mounting the PVC and writing order records.

3. **statefulset-database.yaml**
   3-replica StatefulSet with volumeClaimTemplates.

4. **service-headless.yaml**
   Headless Service giving each replica a stable DNS name.

5. **orders.csv**
   Mock order records written to the volume.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCESS MAP

How Lab 8 Runs



THE POINT

Decouple storage from compute using PVCs, StorageClasses and StatefulSets, and model a service mesh's role.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCEDURE

Persist Data with PersistentVolumes, StatefulSets and Storage Classes — Part 1/3

1

Inspect the StorageClasses the cluster offers for dynamic provisioning

```
$ kubectl get storageclass
```

2

Request storage with a PersistentVolumeClaim

```
$ kubectl apply -f pvc-orders.yaml && kubectl get pvc -n kcna-lab
```

3

Confirm a PersistentVolume was provisioned automatically to satisfy it

```
$ kubectl get pv
```

4

Run a Pod that mounts the claim and writes the mock order records

```
$ kubectl apply -f pod-with-pvc.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCEDURE

Persist Data with PersistentVolumes, StatefulSets and Storage Classes — Part 2/3

5

Verify the data was written to the persistent volume

```
$ kubectl exec -n kcna-lab writer -- cat /data/orders.csv | head -5
```

6

Delete the Pod — the volume and its data must survive

```
$ kubectl delete pod writer -n kcna-lab
```

7

Recreate the Pod and confirm the data is still there

```
$ kubectl apply -f pod-with-pvc.yaml && kubectl exec -n kcna-lab writer -- wc -l /data/orders.csv
```

8

Deploy a StatefulSet with a headless Service for stable identities

```
$ kubectl apply -f service-headless.yaml -f statefulset-database.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCEDURE

Persist Data with PersistentVolumes, StatefulSets and Storage Classes — Part 3/3

9

Observe ordinal Pod names and the PVC created per replica

```
$ kubectl get pods,pvc -n kcna-lab -l app=database
```

10

Resolve a single replica by its stable DNS name

```
$ kubectl run dns-test -n kcna-lab --rm -it --image=busybox:1.36 --restart=Never -- nslookup da...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · VERIFY

Persist Data with PersistentVolumes, StatefulSets and Storage Classes

✅ Expected result

The order records survive deleting and recreating the Pod, and the StatefulSet produces database-0/1/2 each with its own PVC, with database-0 resolvable by its stable DNS name.

IF IT DOESN'T WORK

PVC stays Pending

No default StorageClass. Check kubectl get storageclass.

Data lost after delete

You deleted the PVC, not just the Pod. The volume follows the claim.

StatefulSet Pods stuck

They start in order — Pod 0 must be Ready before Pod 1 is created.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Container Orchestration and Kubernetes Operations

- You can now: Build an OCI image, run it as a container, and explain the runtime layer beneath Kubernetes.
- You can now: Apply Kubernetes networking to give stable discovery and external access to ephemeral Pods.
- You can now: Apply the 4C security model using RBAC, ServiceAccounts, Secrets and network segmentation.
- You can now: Decouple storage from compute using PVCs, StorageClasses and StatefulSets, and model a service mesh's role.

© 2026 Tertiary Infotech Academy Pte Ltd

03

TOPIC 03

Cloud Native Architecture and Scalability

Autoscaling · Serverless · Community & Governance · Roles & Personas · Open Standards

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 16%

Key Concepts — Cloud Native Architecture and Scalability

1. Cloud native means building loosely-coupled, resilient, observable and manageable systems that exploit elastic infrastructure — not simply running old software in containers.

2. The microservices trade-off: independent deploy and scale, at the cost of network latency, distributed failure modes and far greater operational complexity.

3. Kubernetes autoscales on three axes: the HPA adds Pod replicas, the VPA right-sizes requests and limits, and the Cluster Autoscaler adds or removes nodes.

4. The HPA computes desired replicas as ceil(current replicas x current metric / target metric), so target selection directly determines cost and headroom.

5. Serverless removes server management and scales to zero; Knative and KEDA bring the event-driven, scale-to-zero model onto Kubernetes itself.

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 16%

Key Concepts — Cloud Native Architecture and Scalability (continued)

1. Resilience is designed in: liveness probes restart a hung container, readiness probes remove a Pod from the Service, and a PodDisruptionBudget protects availability during maintenance.
2. The CNCF stewards the ecosystem with a maturity ladder — Sandbox, Incubating, Graduated — that is a genuine signal of production readiness.
3. Open standards prevent lock-in: OCI for images, CRI for runtimes, CNI for networking, CSI for storage, SMI for meshes and OpenTelemetry for telemetry.
4. Cloud native roles and personas — platform engineer, SRE, DevOps engineer, security engineer — divide responsibility across the same shared platform.
5. The technical blueprint records the decisions: chosen components, their interfaces, non-functional requirements, and the rationale behind each trade-off.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Cloud Native Architecture and Scalability

Lab 9

- Autoscale Workloads with the Horizontal Pod Autoscaler

Lab 10

- Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture

Lab 11

- Explore Serverless, Open Standards and the CNCF Landscape

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Autoscale Workloads with the Horizontal Pod Autoscaler

LAB 9

Identify the technical requirements for elastic scaling and demonstrate autoscaling under real load.

The learner enables the metrics pipeline, deploys a CPU-bound service with the resource requests the HPA needs, applies an HPA, generates load, and records the scale-up and scale-down behaviour as evidence for the blueprint.

YOU'LL BUILD

A working HPA that scales a Deployment from 1 to 10 replicas under load and back down again

TOOLCHAIN

HorizontalPodAutoscaler, metrics-server, resource requests, Deployment, Cluster Autoscaler concepts

DONE WHEN

kubectl get hpa shows TARGETS rising well above 50% and REPLICAS increasing from 1 towards 10 under load, then returning to 1 after the 5-minute scale-down stabilisation window.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 9 — Files in Your Lab Folder

1. **deployment-cpu-load.yaml**
   CPU-bound PHP-Apache workload with CPU requests set.

2. **service-cpu-load.yaml**
   Service fronting the CPU-bound workload.

3. **hpa-cpu.yaml**
   HPA targeting 50% average CPU utilisation, 1-10 replicas.

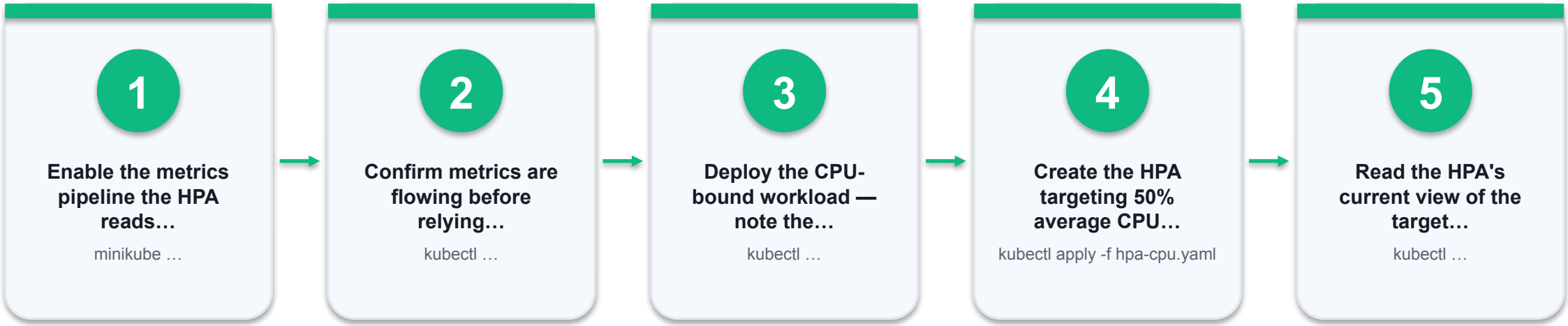
4. **load-generator.yaml**
   Pod generating continuous HTTP load.

5. **scaling-observations.md**
   Worksheet: record replica counts against elapsed time.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · PROCESS MAP

How Lab 9 Runs



THE POINT

Identify the technical requirements for elastic scaling and demonstrate autoscaling under real load.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · PROCEDURE

Autoscale Workloads with the Horizontal Pod Autoscaler — Part 1/3

1

Enable the metrics pipeline the HPA reads from

```
$ minikube addons enable metrics-server
```

2

Confirm metrics are flowing before relying on them

```
$ kubectl top nodes && kubectl top pods -n kcna-lab
```

3

Deploy the CPU-bound workload — note the CPU request the HPA needs

```
$ kubectl apply -f deployment-cpu-load.yaml -f service-cpu-load.yaml
```

4

Create the HPA targeting 50% average CPU utilisation

```
$ kubectl apply -f hpa-cpu.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · PROCEDURE

Autoscale Workloads with the Horizontal Pod Autoscaler — Part 2/3

5

Read the HPA's current view of the target metric

```
$ kubectl get hpa cpu-load -n kcna-lab -o wide
```

6

Start the load generator to drive CPU above the target

```
$ kubectl apply -f load-generator.yaml
```

7

Watch the HPA scale the Deployment up in response

```
$ kubectl get hpa cpu-load -n kcna-lab -w --timeout=300s
```

8

Record the scaling decisions the controller made and why

```
$ kubectl describe hpa cpu-load -n kcna-lab | grep -A10 Events
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · PROCEDURE

Autoscale Workloads with the Horizontal Pod Autoscaler — Part 3/3

9 **Remove the load and observe the stabilisation window before scale-down**

```
$ kubectl delete -f load-generator.yaml
```

10 **Complete scaling-observations.md with your replica-count timeline**

```
$ kubectl get hpa cpu-load -n kcna-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · VERIFY

Autoscale Workloads with the Horizontal Pod Autoscaler

✅ Expected result

kubectl get hpa shows TARGETS rising well above 50% and REPLICAS increasing from 1 towards 10 under load, then returning to 1 after the 5-minute scale-down stabilisation window.

IF IT DOESN'T WORK

HPA shows <unknown>

metrics-server is not ready: minikube addons enable metrics-server, then wait 60s.

No scale-up under load

The Deployment needs a CPU request — the HPA has no denominator without it.

Scale-down seems stuck

That is the 300s stabilisation window. Wait it out.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture

LAB 10

Prepare a technical blueprint for a resilient multi-tier cloud native architecture and prove its failure behaviour.

The learner deploys a three-tier application, configures liveness, readiness and startup probes, deliberately fails a container to watch Kubernetes self-heal, and protects availability during maintenance with a PodDisruptionBudget.

YOU'LL BUILD

A three-tier application with health probes and a PodDisruptionBudget, plus a documented blueprint

TOOLCHAIN

liveness/readiness/startup probes, PodDisruptionBudget, Deployment, Service, resource limits

DONE WHEN

The API container restarts automatically after the health file is removed (RESTARTS increments and a Liveness probe failed event is logged), and kubectl get pdb reports ALLOWED DISRUPTIONS consistent with minAvailable: 2.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 10 — Files in Your Lab Folder

1. **tier-frontend.yaml**
   Frontend Deployment + Service with readiness probe.

2. **tier-api.yaml**
   API tier with liveness, readiness and startup probes.

3. **tier-database.yaml**
   Database tier with a persistent volume.

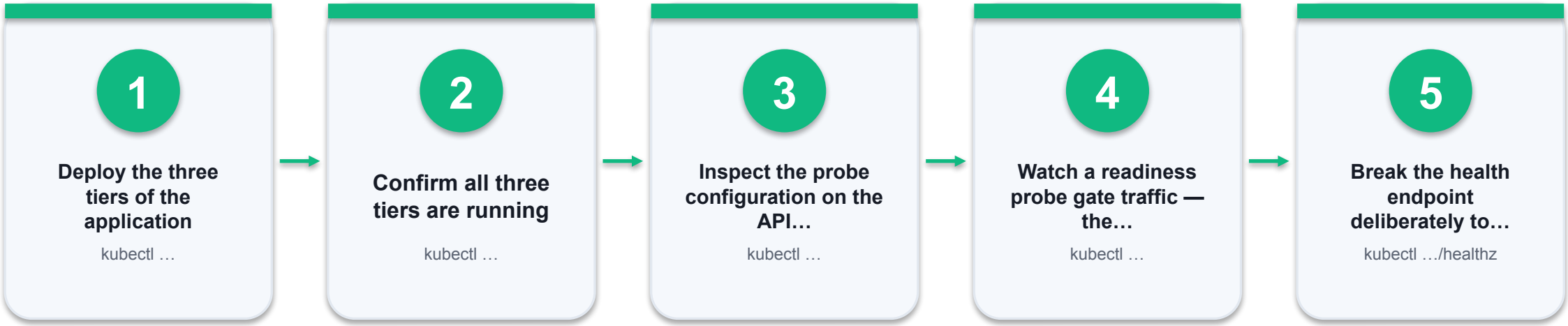
4. **pdb-api.yaml**
   PodDisruptionBudget requiring minAvailable: 2.

5. **blueprint-template.md**
   Technical blueprint worksheet — components, interfaces, NFRs, trade-offs.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · PROCESS MAP

How Lab 10 Runs



THE POINT

Prepare a technical blueprint for a resilient multi-tier cloud native architecture and prove its failure behaviour.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · PROCEDURE

Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture — Part 1/3

1

Deploy the three tiers of the application

```
$ kubectl apply -f tier-database.yaml -f tier-api.yaml -f tier-frontend.yaml
```

2

Confirm all three tiers are running and their Services resolve

```
$ kubectl get deploy,svc,pods -n kcna-lab -l tier
```

3

Inspect the probe configuration on the API tier

```
$ kubectl get deploy api -n kcna-lab -o jsonpath='{.spec.template.spec.containers[0].livenessPr...
```

4

Watch a readiness probe gate traffic — the Pod is Running but not Ready

```
$ kubectl get pods -n kcna-lab -l tier=api -o wide
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · PROCEDURE

Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture — Part 2/3

5

Break the health endpoint deliberately to trigger the liveness probe

```
$ kubectl exec -n kcna-lab deploy/api -- rm -f /usr/share/nginx/html/healthz
```

6

Watch Kubernetes restart the unhealthy container automatically

```
$ kubectl get pods -n kcna-lab -l tier=api -w --timeout=120s
```

7

Confirm the restart count and the probe failure event

```
$ kubectl describe pod -n kcna-lab -l tier=api | grep -A5 'Liveness probe failed'
```

8

Apply a PodDisruptionBudget to protect availability during maintenance

```
$ kubectl apply -f pdb-api.yaml && kubectl get pdb -n kcna-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd

Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture — Part 3/3

9 Attempt a node drain and observe the PDB blocking excessive eviction

```
$ kubectl drain minikube --ignore-daemonsets --delete-emptydir-data --dry-run=server 2>&1 | hea...
```

10 Complete blueprint-template.md documenting components, interfaces and NFRs

```
$ cat blueprint-template.md
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · VERIFY

Build Resilience with Probes, Disruption Budgets and Multi-Tier Architecture

✅ Expected result

The API container restarts automatically after the health file is removed (RESTARTS increments and a Liveness probe failed event is logged), and kubectl get pdb reports ALLOWED DISRUPTIONS consistent with minAvailable: 2.

IF IT DOESN'T WORK

Container never restarts

The liveness probe path still returns 200. Confirm you removed the health file.

Pod Running but not Ready

That is the readiness probe doing its job — check the probe path and port.

PDB shows 0 allowed

Fewer healthy Pods than minAvailable. Scale up before draining.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Explore Serverless, Open Standards and the CNCF Landscape

LAB 11

Identify stakeholder demands and open standards that constrain a cloud native technical blueprint.

The learner examines the open standards that keep Kubernetes portable — OCI, CRI, CNI, CSI — by inspecting the cluster's own implementations, models event-driven scale-to-zero, and maps candidate CNCF projects to the blueprint's requirements.

YOU'LL BUILD

A standards-and-projects selection matrix justifying each component of the blueprint

TOOLCHAIN

CNCF landscape, OCI, CRI, CNI, CSI, Knative and KEDA concepts, ConfigMap, scale-to-zero

DONE WHEN

Your standards-matrix.md names the actual CRI, CNI and CSI implementations found in this cluster, and the scale-to-zero Deployment reports 0/0 replicas when idle and 3/3 after scaling back up.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 11 — Files in Your Lab Folder

1. **standards-matrix.md**
   Worksheet mapping each standard to its cluster implementation.

2. **configmap-scaling-policy.yaml**
   Event-driven scaling policy expressed as config.

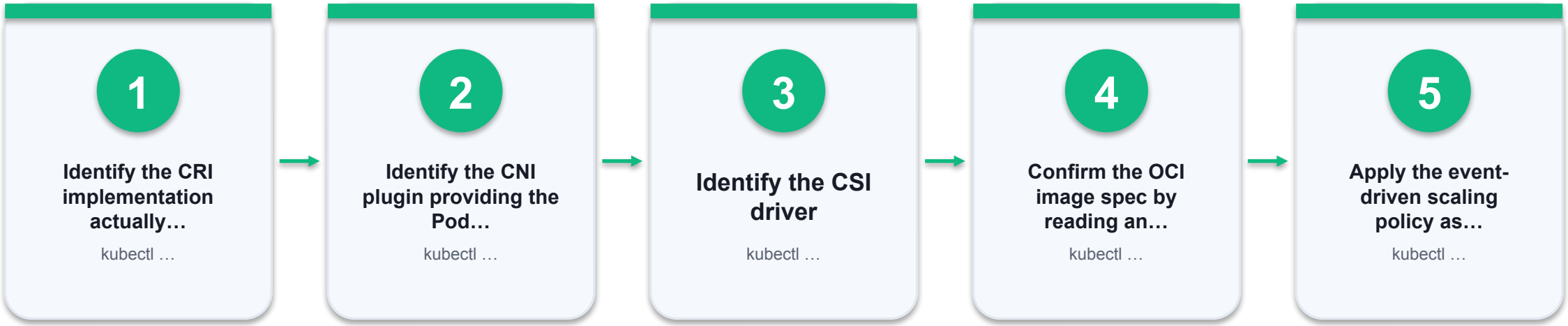
3. **deployment-scale-to-zero.yaml**
   Deployment demonstrating scale-to-zero and back.

4. **cncf-selection.csv**
   Mock evaluation of 8 CNCF projects against blueprint criteria.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · PROCESS MAP

How Lab 11 Runs



THE POINT

Identify stakeholder demands and open standards that constrain a cloud native technical blueprint.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · PROCEDURE

Explore Serverless, Open Standards and the CNCF Landscape — Part 1/3

1

Identify the CRI implementation actually running your containers

```
$ kubectl get nodes -o jsonpath='{.items[*].status.nodeInfo.containerRuntimeVersion}'
```

2

Identify the CNI plugin providing the Pod network

```
$ kubectl get pods -n kube-system -o name | grep -Ei 'cni|calico|flannel|kindnet'
```

3

Identify the CSI driver and StorageClass provisioner in use

```
$ kubectl get storageclass -o jsonpath='{.items[*].provisioner}'
```

4

Confirm the OCI image spec by reading an image's architecture and layers

```
$ kubectl get pod -n kcna-lab -o jsonpath='{.items[0].status.containerStatuses[0].imageID}'
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · PROCEDURE

Explore Serverless, Open Standards and the CNCF Landscape — Part 2/3

5

Apply the event-driven scaling policy as configuration

```
$ kubectl apply -f configmap-scaling-policy.yaml
```

6

Deploy the workload that will demonstrate scale-to-zero

```
$ kubectl apply -f deployment-scale-to-zero.yaml
```

7

Scale the workload to zero to model the serverless idle state

```
$ kubectl scale deploy scale-to-zero -n kcna-lab --replicas=0
```

8

Confirm no Pods and therefore no compute cost while idle

```
$ kubectl get deploy scale-to-zero -n kcna-lab
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · PROCEDURE

Explore Serverless, Open Standards and the CNCF Landscape — Part 3/3

9 Scale back up on demand and measure the cold-start latency

```
$ time kubectl scale deploy scale-to-zero -n kcna-lab --replicas=3 --timeout=60s
```

10 Complete cncf-selection.csv and standards-matrix.md for your blueprint

```
$ cat cncf-selection.csv
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · VERIFY

Explore Serverless, Open Standards and the CNCF Landscape

✅ Expected result

Your standards-matrix.md names the actual CRI, CNI and CSI implementations found in this cluster, and the scale-to-zero Deployment reports 0/0 replicas when idle and 3/3 after scaling back up.

IF IT DOESN'T WORK

No CNI pod listed

minikube may use kindnet. Widen the grep or check kubectl get pods -n kube-system.

Scale to zero rejected

Some controllers block 0. Use kubectl scale --replicas=0 on the Deployment itself.

Cold start feels slow

That is the trade-off being taught — the image pull dominates the first start.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Cloud Native Architecture and Scalability

- You can now: Identify the technical requirements for elastic scaling and demonstrate autoscaling under real load.
- You can now: Prepare a technical blueprint for a resilient multi-tier cloud native architecture and prove its failure behaviour.
- You can now: Identify stakeholder demands and open standards that constrain a cloud native technical blueprint.

© 2026 Tertiary Infotech Academy Pte Ltd

04

TOPIC 04

Kubernetes Monitoring and Observability

Telemetry & Observability · Prometheus · Cost Management

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 8%

Key Concepts — Kubernetes Monitoring and Observability

1. Monitoring answers questions you already knew to ask; observability lets you ask new questions of a running system without shipping new code.

2. The three pillars are metrics (cheap, aggregatable numbers), logs (discrete events with detail) and traces (one request's path across services).

3. Prometheus pulls metrics by scraping an HTTP /metrics endpoint on a schedule, and stores them as a time series identified by a name and a set of labels.

4. The four Prometheus metric types: counter (only increases), gauge (goes up and down), histogram (bucketed observations) and summary (client-side quantiles).

5. PromQL selects a series then applies functions — rate() over a counter gives per-second throughput; histogram_quantile() over buckets gives latency percentiles.

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 8%

Key Concepts — Kubernetes Monitoring and Observability (continued)

1. The four golden signals — latency, traffic, errors and saturation — are the minimum viable dashboard for any service.
2. Alert on symptoms that users feel, not on causes: page on the error-rate SLO burning, not on a single Pod restarting.
3. OpenTelemetry is the vendor-neutral standard for generating and exporting metrics, logs and traces, so instrumentation survives a change of backend.
4. Cost management is an observability problem: unbounded requests, idle nodes and over-provisioned limits are all visible in the same metrics.
5. FinOps practices — right-sizing from real usage, cluster autoscaling, spot nodes and namespace-level showback — turn those metrics into savings.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Kubernetes Monitoring and Observability

Lab 12

- Instrument and Collect Metrics with Prometheus

Lab 13

- Apply the Golden Signals, Alerting and Cost Management

—

- —

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON

Instrument and Collect Metrics with Prometheus

LAB 12

Demonstrate how observability components collect the telemetry needed to address a business problem.

The learner deploys Prometheus into the cluster, exposes an application /metrics endpoint carrying mock order and latency data, confirms the scrape targets are healthy, and queries the data with PromQL to answer a stated business question.

YOU'LL BUILD

A running Prometheus scraping a custom application exporter, queried with PromQL

TOOLCHAIN

Prometheus, exporters, /metrics endpoint, PromQL, ServiceMonitor concepts, counters/gauges/histograms

DONE WHEN

The Prometheus targets API reports health=up for every job, and the PromQL queries return numeric values for order throughput and 95th-percentile latency.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 12 — Files in Your Lab Folder

1. **prometheus-config.yaml**
   ConfigMap with the Prometheus scrape configuration.

2. **prometheus-deployment.yaml**
   Prometheus Deployment + Service.

3. **prometheus-rbac.yaml**
   ServiceAccount and ClusterRole for service discovery.

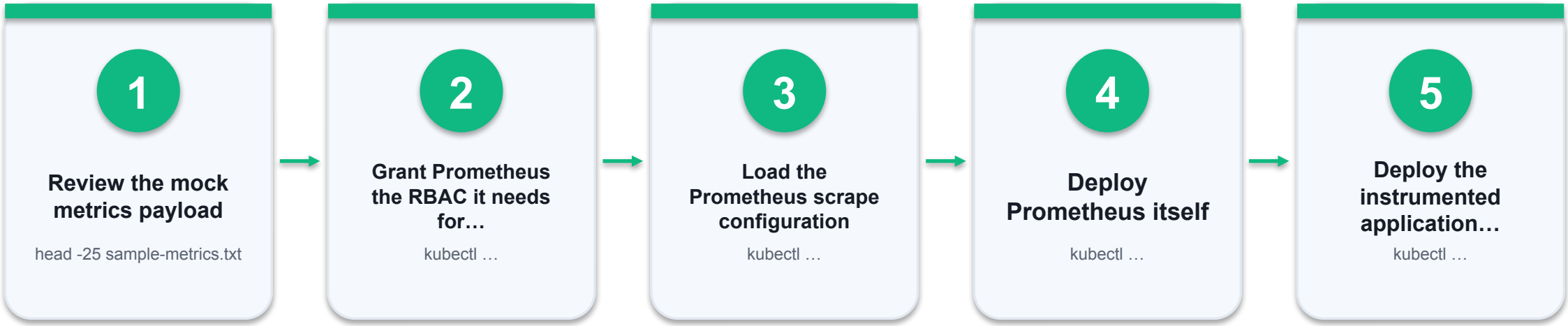
4. **metrics-app.yaml**
   Sample app exposing counters, gauges and histograms.

5. **sample-metrics.txt**
   Mock /metrics output — orders, errors and latency buckets.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · PROCESS MAP

How Lab 12 Runs



THE POINT

Demonstrate how observability components collect the telemetry needed to address a business problem.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · PROCEDURE

Instrument and Collect Metrics with Prometheus — Part 1/3

1

Review the mock metrics payload and identify each metric type

```
$ head -25 sample-metrics.txt
```

2

Grant Prometheus the RBAC it needs for Kubernetes service discovery

```
$ kubectl apply -f prometheus-rbac.yaml
```

3

Load the Prometheus scrape configuration

```
$ kubectl apply -f prometheus-config.yaml
```

4

Deploy Prometheus itself

```
$ kubectl apply -f prometheus-deployment.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · PROCEDURE

Instrument and Collect Metrics with Prometheus — Part 2/3

5 **Deploy the instrumented application exposing /metrics**

```
$ kubectl apply -f metrics-app.yaml
```

6 **Confirm the application is exposing metrics in the Prometheus text format**

```
$ kubectl exec -n kcna-lab deploy/metrics-app -- wget -qO- localhost:8080/metrics | head -15
```

7 **Port-forward the Prometheus UI to your workstation**

```
$ kubectl port-forward -n kcna-lab svc/prometheus 9090:9090 &
```

8 **Verify every scrape target is UP before trusting the data**

```
$ curl -s 'http://localhost:9090/api/v1/targets' | python3 -c "import json,sys; [print(t['label...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · PROCEDURE

Instrument and Collect Metrics with Prometheus — Part 3/3

9

Query per-second order throughput with rate() over the counter

```
$ curl -s --data-urlencode 'query=rate(orders_total[5m])' http://localhost:9090/api/v1/query
```

10

Compute the 95th-percentile latency from the histogram buckets

```
$ curl -s --data-urlencode 'query=histogram_quantile(0.95, rate(request_duration_seconds_bucket...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · VERIFY

Instrument and Collect Metrics with Prometheus

✅ Expected result

The Prometheus targets API reports health=up for every job, and the PromQL queries return numeric values for order throughput and 95th-percentile latency.

IF IT DOESN'T WORK

Prometheus targets are down

Check the RBAC was applied and the scrape config namespace is kcna-lab.

Port-forward drops

Re-run kubectl port-forward; it does not survive a Pod restart.

PromQL returns empty

Give it two scrape intervals (30s) before the rate() window has data.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON

Apply the Golden Signals, Alerting and Cost Management

LAB 13

Demonstrate how observability and cost data together resolve a specific business problem.

The learner builds the four golden signals for the order service, writes an alerting rule that fires on the symptom rather than the cause, then analyses cost data — requests versus actual usage — and recommends right-sizing backed by evidence.

YOU'LL BUILD

A golden-signals dashboard specification, a firing alert rule, and a costed right-sizing recommendation

TOOLCHAIN

Prometheus alerting rules, golden signals, kubectl top, resource requests, FinOps right-sizing

DONE WHEN

The HighErrorRate alert reaches state=firing in the Prometheus alerts API after the error burst, and your rightsizing-analysis.md names the two most over-provisioned namespaces with the monthly saving each change would deliver.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 13 — Files in Your Lab Folder

1. **alert-rules.yaml**
   Alerting rules for error rate, latency and saturation.

2. **golden-signals.md**
   Worksheet: the PromQL query for each golden signal.

3. **cost-report.csv**
   Mock cost report — 6 namespaces, requested vs used vs monthly cost.

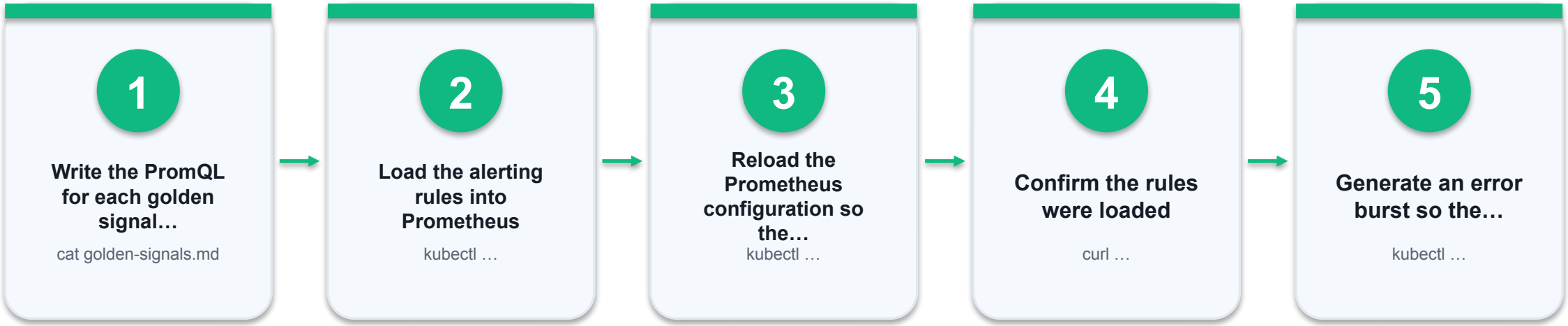
4. **error-burst.yaml**
   Job generating an error burst to make the alert fire.

5. **rightsizing-analysis.md**
   Worksheet for the costed recommendation.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · PROCESS MAP

How Lab 13 Runs



THE POINT

Demonstrate how observability and cost data together resolve a specific business problem.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · PROCEDURE

Apply the Golden Signals, Alerting and Cost Management — Part 1/3

1

Write the PromQL for each golden signal into golden-signals.md

```
$ cat golden-signals.md
```

2

Load the alerting rules into Prometheus

```
$ kubectl apply -f alert-rules.yaml
```

3

Reload the Prometheus configuration so the rules take effect

```
$ kubectl rollout restart deploy/prometheus -n kcna-lab
```

4

Confirm the rules were loaded and are evaluating

```
$ curl -s http://localhost:9090/api/v1/rules | python3 -m json.tool | head -25
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · PROCEDURE

Apply the Golden Signals, Alerting and Cost Management — Part 2/3

5

Generate an error burst so the symptom-based alert fires

```
$ kubectl apply -f error-burst.yaml
```

6

Watch the alert transition from Pending to Firing

```
$ curl -s http://localhost:9090/api/v1/alerts | python3 -m json.tool | head -25
```

7

Compare requested resources against what the Pods actually use

```
$ kubectl top pods -n kcna-lab --sort-by=cpu
```

8

Read the mock cost report and identify the two worst-value namespaces

```
$ column -s, -t cost-report.csv
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · PROCEDURE

Apply the Golden Signals, Alerting and Cost Management — Part 3/3

9 Calculate the waste from over-requested CPU and memory

```
$ python3 -c "import csv;r=list(csv.DictReader(open('cost-report.csv')));[print(x['namespace'],...
```

10 Record your costed right-sizing recommendation with the evidence

```
$ cat rightsizing-analysis.md
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · VERIFY

Apply the Golden Signals, Alerting and Cost Management

✅ Expected result

The HighErrorRate alert reaches state=firing in the Prometheus alerts API after the error burst, and your rightsizing-analysis.md names the two most over-provisioned namespaces with the monthly saving each change would deliver.

IF IT DOESN'T WORK

Alert never fires

Rules load on restart: kubectl rollout restart deploy/prometheus -n kcna-lab.

kubectl top has no data

metrics-server again — enable it and wait for the first scrape.

Alert fires immediately

Check the for: duration; it must stay true for the whole window.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Kubernetes Monitoring and Observability

- You can now: Demonstrate how observability components collect the telemetry needed to address a business problem.
- You can now: Demonstrate how observability and cost data together resolve a specific business problem.

© 2026 Tertiary Infotech Academy Pte Ltd

05

TOPIC 05

Kubernetes Application Deployment and Management

Application Delivery Fundamentals · GitOps · CI/CD

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 8%

Key Concepts — Kubernetes Application Deployment and Management

1. Application delivery on Kubernetes is the practice of moving a change from a commit to running Pods safely, repeatably and reversibly.

2. A Deployment's RollingUpdate strategy with maxSurge and maxUnavailable is the primitive behind every zero-downtime release.

3. Progressive delivery reduces blast radius: blue-green swaps traffic at once, canary shifts a small percentage first and watches the metrics.

4. kubectl rollout status, history and undo are the operational loop — every rollout must be reversible before it is attempted.

5. Packaging manifests matters: Helm templates a chart with values, while Kustomize overlays patches onto a plain base with no templating language.

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA EXAM WEIGHTING 8%

Key Concepts — Kubernetes Application Deployment and Management (continued)

1. GitOps makes Git the single source of truth: the desired state is declared in a repository and an in-cluster agent continuously reconciles the cluster to it.

2. Because reconciliation is continuous, GitOps also corrects configuration drift — a manual kubectl edit is reverted automatically.

3. Argo CD and Flux are the two CNCF-graduated GitOps engines; both watch a repo, diff against the cluster and apply the difference.

4. A CI/CD pipeline builds and tests the image, then updates the manifest repository — CI produces artifacts, CD reconciles state.

5. A system review closes the loop: read the observability data against the SLOs, decide what to change, and adapt the architecture accordingly.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Kubernetes Application Deployment and Management

Lab 14

- Perform Zero-Downtime Rolling Updates and Canary Releases

Lab 15

- Package Applications with Helm and Kustomize

Lab 16

- Automate Delivery with GitOps and CI/CD, and Conduct a System Review

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON

Perform Zero-Downtime Rolling Updates and Canary Releases

LAB 14

Implement a controlled release and roll it back, monitoring solution status throughout.

The learner tunes a Deployment's rolling-update strategy, releases a new version while continuously probing for downtime, deliberately deploys a broken image to see the rollout stall, rolls back, then runs a weighted canary release.

YOU'LL BUILD

A verified zero-downtime rolling update, a recovered failed rollout, and a working canary split

TOOLCHAIN

Deployment strategy, maxSurge/maxUnavailable, readiness probes, kubectl rollout, canary via replica weighting

DONE WHEN

uptime.log records zero failed requests across the rolling update, the broken rollout is detected and reversed by kubectl rollout undo, and the canary Service lists endpoints from both Deployments before promotion.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 14 — Files in Your Lab Folder

1. **deployment-v1.yaml**
   Version 1 with a tuned RollingUpdate strategy.

2. **deployment-v2-broken.yaml**
   Version 2 with an invalid image — the rollout stalls.

3. **canary-stable.yaml**
   Stable Deployment, 9 replicas, shared Service selector.

4. **canary-new.yaml**
   Canary Deployment, 1 replica — a 10% traffic split.

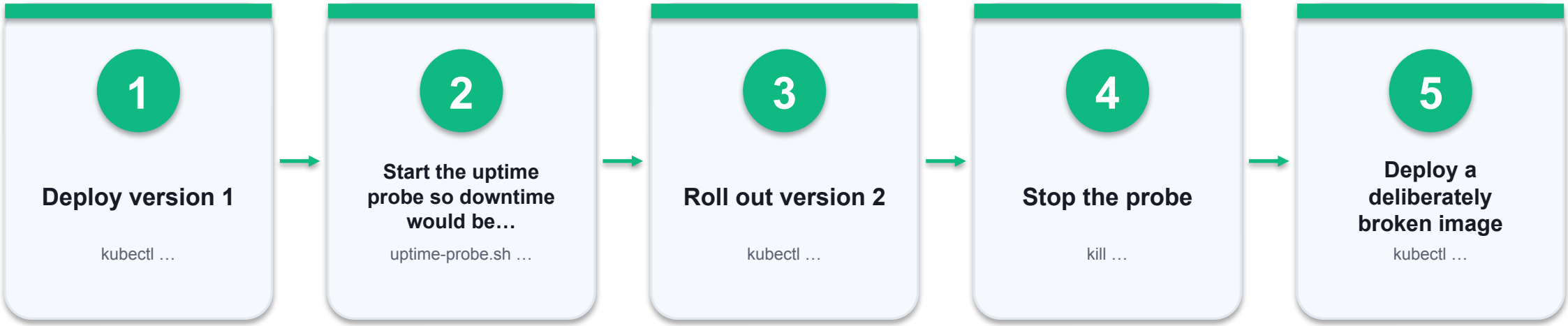
5. **service-canary.yaml**
   Service selecting both stable and canary Pods.

6. **uptime-probe.sh**
   Loop that records any failed request during the release.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · PROCESS MAP

How Lab 14 Runs



THE POINT

Implement a controlled release and roll it back, monitoring solution status throughout.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · PROCEDURE

Perform Zero-Downtime Rolling Updates and Canary Releases — Part 1/3

1

Deploy version 1 and review its rolling-update strategy

```
$ kubectl apply -f deployment-v1.yaml && kubectl get deploy app-release -n kcna-lab -o jsonpath...
```

2

Start the uptime probe so downtime would be recorded

```
$ bash uptime-probe.sh > uptime.log 2>&1 &
```

3

Roll out version 2 and watch replicas cut over incrementally

```
$ kubectl set image deploy/app-release app=nginx:1.27-alpine -n kcna-lab && kubectl rollout sta...
```

4

Stop the probe and confirm no requests failed during the release

```
$ kill %1; grep -c FAIL uptime.log || echo 'zero failures'
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · PROCEDURE

Perform Zero-Downtime Rolling Updates and Canary Releases — Part 2/3

5 **Deploy a deliberately broken image and watch the rollout stall**

```
$ kubectl apply -f deployment-v2-broken.yaml
```

6 **Diagnose why the rollout is stuck rather than guessing**

```
$ kubectl rollout status deploy/app-release -n kcna-lab --timeout=60s; kubectl get pods -n kcna...
```

7 **Roll back to the last good revision and confirm recovery**

```
$ kubectl rollout undo deploy/app-release -n kcna-lab && kubectl rollout status deploy/app-rele...
```

8 **Deploy the stable and canary Deployments behind one Service**

```
$ kubectl apply -f canary-stable.yaml -f canary-new.yaml -f service-canary.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · PROCEDURE

Perform Zero-Downtime Rolling Updates and Canary Releases — Part 3/3

9 Confirm the Service is load-balancing across both versions

```
$ kubectl get endpoints app-canary -n kcna-lab -o jsonpath='{.subsets[0].addresses[*].ip}'
```

10 Measure the actual traffic split, then promote or abort the canary

```
$ kubectl scale deploy app-canary -n kcna-lab --replicas=10 && kubectl scale deploy app-stable ...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · VERIFY

Perform Zero-Downtime Rolling Updates and Canary Releases

✅ Expected result

uptime.log records zero failed requests across the rolling update, the broken rollout is detected and reversed by kubectl rollout undo, and the canary Service lists endpoints from both Deployments before promotion.

IF IT DOESN'T WORK

uptime.log shows failures

maxUnavailable must be 0 AND a readiness probe must be set.

Broken rollout never fails

Set progressDeadlineSeconds so it gives up instead of hanging.

Canary gets no traffic

Both Deployments must carry the shared label the Service selects on.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON

Package Applications with Helm and Kustomize

LAB 15

Package and parameterise a solution so it can be delivered repeatably across environments.

The learner packages the application as a Helm chart with environment-specific values, installs and upgrades it, then achieves the same result with Kustomize overlays and compares the two approaches for the delivery blueprint.

YOU'LL BUILD

An installed and upgraded Helm release plus equivalent dev and prod Kustomize overlays

TOOLCHAIN

Helm (chart, values, release, upgrade, rollback), Kustomize (base, overlays, patches), kubectl apply -k

DONE WHEN

helm list shows the webapp release at revision 3 after the rollback, and the dev and prod Kustomize overlays render 1 and 5 replicas respectively from the same base.

© 2026 Tertiary Infotech Academy Pte Ltd

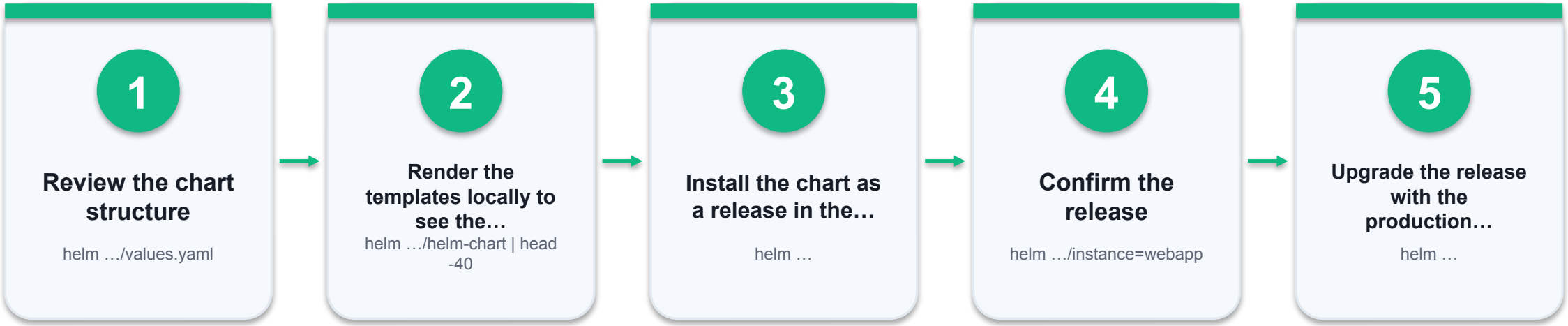
Lab 15 — Files in Your Lab Folder

1. **helm-chart/Chart.yaml**
   Helm chart metadata.
2. **helm-chart/values.yaml**
   Default values — replicas, image, resources.
3. **helm-chart/values-prod.yaml**
   Production overrides — 5 replicas, higher limits.
4. **helm-chart/templates/deployment.yaml**
   Templated Deployment.
5. **helm-chart/templates/service.yaml**
   Templated Service.
6. **kustomize/base/kustomization.yaml**
   Kustomize base.
7. **kustomize/overlays/dev/kustomization.yaml**
   Dev overlay — 1 replica.
8. **kustomize/overlays/prod/kustomization.yaml**
   Prod overlay — 5 replicas + patch.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · PROCESS MAP

How Lab 15 Runs



THE POINT

Package and parameterise a solution so it can be delivered repeatably across environments.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · PROCEDURE

Package Applications with Helm and Kustomize — Part 1/3

1

Review the chart structure and where values are substituted

```
$ helm lint ./helm-chart && cat helm-chart/values.yaml
```

2

Render the templates locally to see the generated manifests

```
$ helm template webapp ./helm-chart | head -40
```

3

Install the chart as a release in the cluster

```
$ helm install webapp ./helm-chart --namespace kcna-lab
```

4

Confirm the release and the resources it created

```
$ helm list -n kcna-lab && kubectl get all -n kcna-lab -l app.kubernetes.io/instance=webapp
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · PROCEDURE

Package Applications with Helm and Kustomize — Part 2/3

5

Upgrade the release with the production values file

```
$ helm upgrade webapp ./helm-chart -f helm-chart/values-prod.yaml -n kcna-lab
```

6

Verify the upgrade took effect and review the release history

```
$ kubectl get deploy -n kcna-lab -l app.kubernetes.io/instance=webapp && helm history webapp -n...
```

7

Roll the release back to revision 1

```
$ helm rollback webapp 1 -n kcna-lab
```

8

Build the dev overlay with Kustomize and inspect the output

```
$ kubectl kustomize kustomize/overlays/dev | head -30
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · PROCEDURE

Package Applications with Helm and Kustomize — Part 3/3

9

Apply the prod overlay and compare it with the dev result

```
$ kubectl apply -k kustomize/overlays/prod
```

10

Compare the two approaches and record which suits your blueprint

```
$ diff <(kubectl kustomize kustomize/overlays/dev) <(kubectl kustomize kustomize/overlays/prod)...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · VERIFY

Package Applications with Helm and Kustomize

✅ Expected result

helm list shows the webapp release at revision 3 after the rollback, and the dev and prod Kustomize overlays render 1 and 5 replicas respectively from the same base.

IF IT DOESN'T WORK

helm lint fails

Run it against the chart directory, not a template file.

Release already exists

Upgrade instead of install, or helm uninstall webapp first.

Kustomize build errors

Check the overlay's resources path actually points at ../../base.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON

Automate Delivery with GitOps and CI/CD, and Conduct a System Review

LAB 16

Implement regular system reviews to monitor solution status and adapt within a cloud native framework.

The learner models the GitOps reconciliation loop with a Git-backed manifest repository, deliberately introduces configuration drift to watch reconciliation correct it, walks a CI/CD pipeline definition, then conducts a structured system review against the SLOs and records the adaptations.

YOU'LL BUILD

A reconciling GitOps repository that self-corrects drift, plus a completed system review report

TOOLCHAIN

GitOps, Argo CD / Flux concepts, Git, CI/CD pipeline, reconciliation loop, SLOs, system review

DONE WHEN

After the manual scale to 7, the reconciliation loop restores the replica count declared in Git; after committing replicas: 4, the same loop rolls the change out — and your system-review-report.md lists at least three evidence-backed adaptations against the breached SLOs.

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 16 — Files in Your Lab Folder

1. **gitops-repo/manifests/deployment.yaml**
   The declared desired state in Git.

2. **gitops-repo/manifests/service.yaml**
   Service manifest under version control.

3. **reconcile.sh**
   Reconciliation loop simulating a GitOps agent.

4. **pipeline.yaml**
   CI/CD pipeline definition — build, test, scan, update manifest.

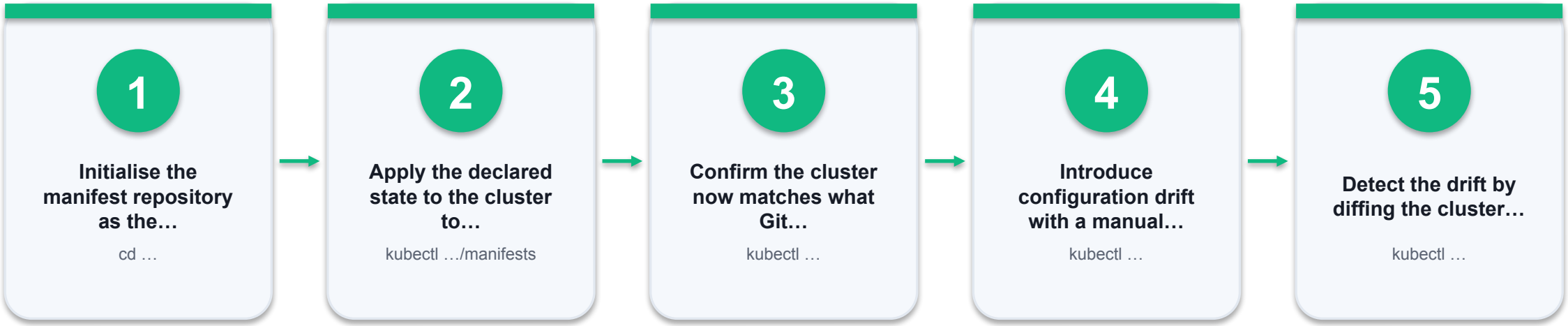
5. **slo-targets.csv**
   Mock SLO targets and 30-day actuals for 5 services.

6. **system-review-report.md**
   Structured system review template.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · PROCESS MAP

How Lab 16 Runs



THE POINT

Implement regular system reviews to monitor solution status and adapt within a cloud native framework.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · PROCEDURE

Automate Delivery with GitOps and CI/CD, and Conduct a System Review — Part 1/3

1

Initialise the manifest repository as the single source of truth

```
$ cd gitops-repo && git init && git add . && git commit -m 'Declare desired state'
```

2

Apply the declared state to the cluster to establish the baseline

```
$ kubectl apply -f gitops-repo/manifests/
```

3

Confirm the cluster now matches what Git declares

```
$ kubectl get deploy gitops-app -n kcna-lab -o jsonpath='{.spec.replicas}'
```

4

Introduce configuration drift with a manual out-of-band change

```
$ kubectl scale deploy gitops-app -n kcna-lab --replicas=7
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · PROCEDURE

Automate Delivery with GitOps and CI/CD, and Conduct a System Review — Part 2/3

5

Detect the drift by diffing the cluster against the repository

```
$ kubectl diff -f gitops-repo/manifests/deployment.yaml || true
```

6

Run the reconciliation loop and watch the drift be corrected

```
$ bash reconcile.sh
```

7

Confirm the cluster was returned to the declared state automatically

```
$ kubectl get deploy gitops-app -n kcna-lab -o jsonpath='{.spec.replicas}'
```

8

Change the desired state in Git and let reconciliation roll it out

```
$ cd gitops-repo && sed -i.bak 's/replicas: 3/replicas: 4/' manifests/deployment.yaml && git co...
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · PROCEDURE

Automate Delivery with GitOps and CI/CD, and Conduct a System Review — Part 3/3

9

Walk the CI/CD pipeline and identify which stage is CI and which is CD

```
$ cat pipeline.yaml
```

10

Conduct the system review against the SLOs and record the adaptations

```
$ column -s, -t slo-targets.csv && cat system-review-report.md
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · VERIFY

Automate Delivery with GitOps and CI/CD, and Conduct a System Review

✅ Expected result

After the manual scale to 7, the reconciliation loop restores the replica count declared in Git; after committing replicas: 4, the same loop rolls the change out — and your system-review-report.md lists at least three evidence-backed adaptations against the breached SLOs.

IF IT DOESN'T WORK

kubectl diff exits non-zero

That is expected — a non-zero exit means drift was found.

Reconcile does not revert

Run it from the lab folder so gitops-repo/manifests resolves.

git commit rejected

Set an identity: git config user.email and user.name in the repo.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Kubernetes Application Deployment and Management

- You can now: Implement a controlled release and roll it back, monitoring solution status throughout.
- You can now: Package and parameterise a solution so it can be delivered repeatably across environments.
- You can now: Implement regular system reviews to monitor solution status and adapt within a cloud native framework.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Continue Your Learning

1

Official documentation

https://kubernetes.io/docs/ — the reference for every object and field you met this course.

2

Sit the KCNA exam

This course is aligned to the five CNCF KCNA domains and their weightings — book while it is fresh.

3

Rebuild the labs

Redo each lab from an empty folder until writing the manifests is automatic.

4

Run a real cluster

Move from minikube to a managed cluster — EKS, AKS or GKE — and repeat Topics 3 to 5.

5

Adopt GitOps

Put your manifests in Git and install Argo CD or Flux to reconcile them for real.

6

Join the community

CNCF Slack, KubeCon talks and the Kubernetes SIGs are all open to you.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTINUE WITH TERTIARY INFOTECH

Recommended Courses

1. **Docker Container Management — build, ship and run containers with confidence.**
2. **Certified Kubernetes Administrator (CKA) — take your cluster operations to production depth.**
3. **DevOps with CI/CD Pipelines — automate build, test and deployment end to end.**
4. **AWS / Azure Cloud Architect — run cloud native workloads on managed Kubernetes.**
5. **Terraform Infrastructure as Code — provision the clusters your applications run on.**

© 2026 Tertiary Infotech Academy Pte Ltd

WE'RE HERE TO HELP

Support

1

Email

enquiry@tertiaryinfotech.com

2

Telephone

+65 6100 0613

3

Website

www.tertiarycourses.com.sg

4

LMS / TMS

https://lms-tms.tertiaryinfotech.com

5

Course page

https://www.tertiarycourses.com.sg/kubernetes-fundamentals-for-beginners.html

6

Lab repository

https://github.com/tertiarycourses/C742-Kubernetes-Fudamentals-for-Beginners

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE YOU LEAVE

Course Feedback

1

Share your feedback

Please complete the course feedback form before you leave — it helps us improve this course.

2

Tell us what worked

Let us know which labs and topics helped most, and what we could do better.

3

Certificate of completion

Your certificate of completion will be emailed to you after the course.

4

Keep in touch

Questions after the course? Email enquiry@tertiaryinfotech.com.

© 2026 Tertiary Infotech Academy Pte Ltd

HAPPY ORCHESTRATING

Thank You!

You can now design, deploy, observe and operate containerised applications on Kubernetes using cloud native architecture.

© 2026 Tertiary Infotech Academy Pte Ltd